



VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

BRNO UNIVERSITY OF TECHNOLOGY

FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

FACULTY OF INFORMATION TECHNOLOGY

ÚSTAV POČÍTAČOVÉ GRAFIKY A MULTIMÉDIÍ

DEPARTMENT OF COMPUTER GRAPHICS AND MULTIMEDIA

MOBILNÍ APLIKACE PRO ZAPISOVÁNÍ TERMÍNŮ KONZULTACÍ

MOBILE APP FOR SCHEDULING CONSULTATIONS

BAKALÁŘSKÁ PRÁCE

BACHELOR'S THESIS

AUTOR PRÁCE

AUTHOR

MARTIN HANÁK

VEDOUCÍ PRÁCE

SUPERVISOR

prof. Ing. ADAM HEROUT, Ph.D.

BRNO 2026

Zadání bakalářské práce



169488

Ústav: Ústav počítačové grafiky a multimédií (UPGM)
Student: **Hanák Martin**
Program: Informační technologie
Název: **Mobilní aplikace pro zapisování termínů konzultací**
Kategorie: Uživatelská rozhraní
Akademický rok: 2025/26

Zadání:

1. Seznamte se s problematikou návrhu a vývoje moderních mobilních aplikací.
2. Definujte požadavky na mobilní aplikaci pro zapisování termínů konzultací a důležité případy jejího užití.
3. Navrhněte uživatelské rozhraní mobilní aplikace pro zapisování konzultací. Postupujte v součinnosti s jiným studentem, který řeší backend aplikace.
4. Implementujte dílčí prvky uživatelského rozhraní pro zvolenou mobilní platformu.
5. Uživatelské rozhraní testujte s vhodnými respondenty a iterativně vylepšujte.
6. Integrujte dílčí prvky uživatelského rozhraní do aplikace schopné praktického testování a následného nasazení.
7. Zhodnot'te dosažené výsledky a navrhněte možnosti pokračování projektu; vytvořte plakátek a krátké video pro prezentování projektu.

Literatura:

- Elisa Păduraru, Fundamentals of Creating a Great UI/UX, Creative Tim, 2022
- Tidwell et al.: Designing Interfaces: Patterns for Effective Interaction Design, O'Reilly, 2020
- Steve Krug: Don't Make Me Think, Revisited: A Common Sense Approach to Web Usability, ISBN: 978-0321965516
- Steve Krug: Rocket Surgery Made Easy: The Do-It-Yourself Guide to Finding and Fixing Usability, ISBN: 978-0321657299
- Daniel Knott: Hands-On Mobile App Testing: A Guide for Mobile Testers and Anyone Involved in the Mobile App Business, Addison-Wesley Professional, 2015
- Lumpapun Punchoojit, Nuttanont Hongwarittorn: Usability Studies on Mobile User Interface Design Patterns: A Systematic Literature Review, Advances in Human-Computer Interaction, 2017
- A Biørn-Hansen et al.: A Survey and Taxonomy of Core Concepts and Research Challenges in Cross-Platform Mobile Development, ACM Computing Surveys, Vol. 51, Issue 5, September 2019

Při obhajobě semestrální části projektu je požadováno:
body 1.–3., značné rozpracování bodů 4., 5. a 6.

Podrobné závazné pokyny pro vypracování práce viz <https://www.fit.vut.cz/study/theses/>

Vedoucí práce: **Herout Adam, prof. Ing., Ph.D.**
Vedoucí ústavu: Černocký Jan, prof. Dr. Ing.
Datum zadání: 1.11.2025
Termín pro odevzdání: 13.5.2026
Datum schválení: 25.11.2025

Abstrakt

Táto bakalárska práca sa zaoberá návrhom a vývojom mobilnej aplikácie „Office Hours“, ktorá slúži na efektívne plánovanie a správu konzultačných hodín v akademickom prostredí. Práca priamo reaguje na obmedzenia a nedostatky zastaraných systémov, ktoré trpia nepriateľným používateľským rozhraním či nedostatočnou ochranou rezervácií pred nežiaducimi úpravami. Zároveň reflektuje fakt, že bežne dostupné všeobecné rezervačné systémy nedokážu plne vyhovovať špecifickým akademickým procesom. Cieľom práce bolo navrhnúť moderné a bezpečné riešenie pre mobilné zariadenia. Študentom aplikácia umožňuje rezerváciu a rušenie termínov v špecifických konzultačných miestnostiach, po bezpečnom overení e-mailu prostredníctvom jednorazového kódu (OTP). Vyučujúcim systém poskytuje nástroje na organizáciu slotov, blokov a miestností. Klientska časť je implementovaná v multiplatformovom frameworku Flutter s využitím architektonického vzoru MVVM (Model-View-ViewModel). Výsledkom práce je kompletne dokončená, otestovaná a plne funkčná aplikácia, ktorá komunikuje s dedikovaným API a je verejne dostupná na stiahnutie v obchode Google Play.

Abstract

This bachelor's thesis focuses on the design and development of the “Office Hours” mobile application, which is used for the efficient scheduling and management of office hours in an academic setting. The thesis directly addresses the limitations and shortcomings of outdated systems, which suffer from unfriendly user interfaces and insufficient protection of reservations against unauthorized modifications. At the same time, it reflects the fact that commonly available general reservation systems are unable to fully accommodate specific academic processes. The goal of the thesis was to design a modern and secure solution for mobile devices. The application allows students to book and cancel appointments in specific consultation rooms after secure email verification via a one-time password (OTP). The system provides instructors with tools for organizing slots, blocks and rooms. The client-side is implemented in the cross-platform Flutter framework using the MVVM (Model-View-ViewModel) architectural pattern. The result of this work is a completed, tested, and fully functional application that communicates with a dedicated API and is publicly available for download on the Google Play Store.

Klíčové slová

mobilná aplikácia, Flutter, rezervačný systém, konzultačné hodiny, MVVM, plánovanie času, Android, UI/UX

Keywords

mobile application, Flutter, booking system, office hours, MVVM, time management, Android, UI/UX

Citácia

HANÁK, Martin. *Mobilní aplikace pro zapisování termínů konzultací*. Brno, 2026. Bakalářská práce. Vysoké učení technické v Brně, Fakulta informačních technologií. Vedoucí práce prof. Ing. Adam Herout, Ph.D.

Mobilní aplikace pro zapisování termínů konzultací

Prehlásenie

Prehlasujem, že túto bakalársku prácu som vypracoval samostatne pod vedením pána profesora Ing. Adama Herouta, Ph.D. Uviedol som všetky literárne pramene, publikácie a ďalšie zdroje, z ktorých som čerpal. Na jazykovú revíziu technickej správy som použil nástroj Perplexity. Pri tvorbe odovzdaného programového kódu som na odstraňovanie chýb a kontrolu kódu využíval nástroj Claude. Všetky rozhodnutia týkajúce sa návrhu a konečnej implementácie sú moje vlastné.



.....
Martin Hanák
12. mája 2026

Podakovanie

Chcel by som sa poďakovať vedúcemu mojej bakalárskej práce, prof. Ing. Adamovi Heroutovi, Ph.D., za jeho ochotu a odborné vedenie pri vypracovávaní tejto práce.

Obsah

1	Úvod	2
2	Analýza existujúcich rezervačných systémov a špecifikácia požiadaviek	3
2.1	Analýza existujúcich riešení	3
2.2	Špecifikácia požiadaviek na systém	8
3	Použité technológie, nástroje a architektonický vzor	11
3.1	Použité technológie	11
3.2	Architektonický vzor MVVM	13
4	Návrh používateľského rozhrania a štruktúry aplikácie	16
4.1	Návrh používateľského rozhrania	16
4.2	Návrh jednotlivých častí aplikácie	18
4.3	Návrh softvérovej architektúry	22
4.4	Možnosti rozšírenia	23
5	Implementácia mobilnej aplikácie vo frameworku Flutter	29
5.1	Prehľad projektu a adresárová štruktúra	29
5.2	Architektúra MVVM v praxi	29
5.3	Implementácia autentifikácie	32
5.4	Komunikácia s API a perzistencia dát	33
5.5	Implementácia konzultácií: miestnosti, bloky a sloty	34
5.6	Navigácia a prepojenie obrazoviek	40
5.7	Znovupoužiteľné widgety a vizuálne prvky	41
6	Testovanie a nasadenie aplikácie	46
6.1	Manuálne testovanie	46
6.2	Používateľské testovanie	46
6.3	Výsledky testovania a implementované zmeny	47
6.4	Nasadenie aplikácie	49
7	Záver	50
	Literatúra	51
A	Adresárová štruktúra projektu	53
B	Prezentačný plagát	54

Kapitola 1

Úvod

Cieľom tejto bakalárskej práce je navrhnuť a implementovať modernú, bezpečnú a multiplatformovú mobilnú aplikáciu „Office Hours“, ktorá rieši dlhodobé problémy s organizáciou konzultačných hodín na Fakulte informačných technológií VUT v Brne. Súčasný systém pre plánovanie stretnutí medzi študentmi a vyučujúcimi trpia absenciou mobilného rozhrania, neoptimalizovanými procesmi a najmä nedostatočným zabezpečením proti nežiaducim úpravám či rušeniu cudzích termínov. Tieto nedostatky sťažujú študentom orientáciu a zbytočne zvyšujú administratívnu záťaž vyučujúcich pri riešení konfliktov.

Výsledná aplikácia tieto prekážky priamo odstraňuje poskytnutím možnosti správy a rezervácie termínov prostredníctvom smartfónu. Systém podporuje dve roly – Návštevníka a Majiteľa – pričom jeden používateľ môže v rôznych miestnostiach vystupovať v oboch. Detailná špecifikácia funkčných požiadaviek pre každú rolu je uvedená v kapitole 2. Systém zamieňa tradičné prihlasovanie heslom za modernú autentifikáciu pomocou jednorazového kódu (OTP). Študenti majú možnosť aktivovať si e-mailové notifikácie, prispôsobiť vizuálny štýl aplikácie a spravovať vlastný profil vrátane nastavení súkromia.

Predmetom tejto práce je vývoj klientskej časti systému. Aby bola výsledná aplikácia spustiteľná na platformách Android aj iOS, bol zvolený *framework Flutter*. Podrobné zdôvodnenie jeho výberu oproti alternatívnym riešeniam je uvedené v kapitole 3. Spracovanie dát a aplikačnej logiky zabezpečuje dedikované REST API, ktorého vývoj však nie je súčasťou tejto práce, ale je predmetom samostatnej bakalárskej práce iného študenta. Paralelne vzniká webová verzia aplikácie ako predmet súbežne riešenej bakalárskej práce inej študentky. Mobilná aplikácia, webové rozhranie aj serverová časť boli počas vývoja úzko koordinované, pričom klientska aplikácia so serverom bezpečne komunikuje prostredníctvom prístupových tokenov.

Kapitola 2

Analýza existujúcich rezervačných systémov a špecifikácia požiadaviek

Táto kapitola analyzuje súčasný stav organizácie konzultačných hodín na Fakulte informačných technológií VUT, identifikuje nedostatky existujúcich riešení a na ich základe špecifikuje požiadavky na novú mobilnú aplikáciu „Office Hours”.

2.1 Analýza existujúcich riešení

2.1.1 Systém Wiki@Merlin

Základným využívaným nástrojom na Fakulte informačných technológií VUT v Brne je systém Wiki@Merlin¹, bežiaci na rovnomennom fakultnom serveri. Ide o platformu, ktorá primárne slúži na zdieľanie študijných materiálov, návodov a organizáciu predmetov. Mnohí vyučujúci tento systém historicky adaptovali aj na vypisovanie a správu konzultačných hodín. Kontrola prístupu je zabezpečená na úrovni celého systému. Pred zobrazením či úpravou stránky so slotmi systém vyžaduje prihlásenie pomocou školského prihlasovacieho mena a hesla.

Proces rezervácie v tomto systéme spočíva v manuálnej úprave textového dokumentu, kde študenti dopisujú svoje mená do predpripravenej tabuľky alebo zoznamu pomocou špecifickej wiki syntaxe. Tento prístup so sebou prináša dva zásadné problémy. Prvým je výrazný diskomfort pri používaní na mobilných zariadeniach. Používateľské rozhranie systému nie je plne prispôbené pre mobilné zariadenia, čo pri úprave textu na malom displeji smartfónu vyžaduje neustále približovanie, posúvanie obrazovky a sťažuje orientáciu vo wiki syntaxi stránky.

Druhým problémom je absencia ochrany jednotlivých rezervácií. Keďže systém nerozlišuje vlastníka konkrétneho riadku v texte, každý používateľ s prístupom k úprave stránky môže (či už omylom alebo zámerne) zmazať či prepísať meno iného študenta. Tento nedostatok môže znižovať dôveru v systém a vyžaduje dodatočnú kontrolu zo strany vyučujúceho.

¹Wiki@Merlin – webový wiki systém určený na zdieľanie informácií týkajúcich sa školských projektov a rôznych návodov užitočných v rámci štúdia na FIT VUT. Dostupné na: <https://merlin.fit.vut.cz/wiki/index.php/Wiki@Merlin>



Obr. 2.1: Používateľské rozhranie systému Wiki@Merlin s rezervačnými blokmi. Ako príklad je zobrazený reálny rezervačný formulár konzultačných hodín prof. Ing. Adama Herouta, Ph.D.

2.1.2 Systém pre správu diplomových prác (Diplomky FIT)

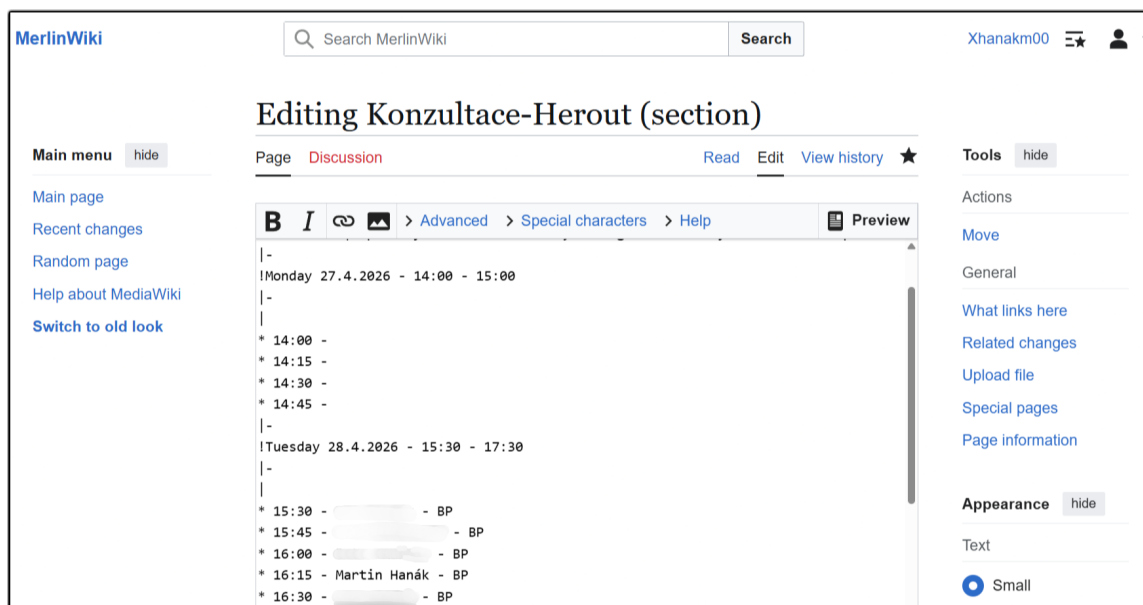
Novšou a sofistikovanejšou alternatívou je fakultný systém Diplomky FIT². Oproti systému Wiki@Merlin predstavuje tento nástroj výrazný posun vpred. Jeho hlavnou výhodou je webové rozhranie a práca so štruktúrovanou databázou, vďaka čomu sú rezervácie termínov a správa dát prehľadnejšie a chránené proti neoprávnenej manipulácii inými používateľmi.

Napriek týmto výhodám má systém z pohľadu modernej prístupnosti aj svoje limity. Prvým z nich je chýbajúca natívna mobilná aplikácia. Hoci webové rozhranie je responzívne, mobilná aplikácia by používateľom poskytla vyšší komfort, rýchlejší prístup a lepšiu integráciu s mobilným operačným systémom bez nutnosti otvárať prehliadač.

Druhým výrazným obmedzením je striktná politika autentifikácie. Prihlásenie do tohto systému je podmienené výlučne vlastníctvom univerzitnej e-mailovej adresy s doménou @fit.vutbr.cz. Napríklad adresy v tvare xlogin00@stud.fit.vut.cz nie sú systémom akceptované. Tento formát predstavuje prekážku pre študentov, ktorí by pri komunikácii a organizácii stretnutí preferovali využitie svojich osobných e-mailových schránok.

Tretím obmedzením je absencia podpory online konzultácií. Systém umožňuje výlučne rezerváciu termínov osobných stretnutí a neposkytuje žiadnu natívnu integráciu s nástrojmi pre videohovory či vzdialenú komunikáciu. Vyučujúci a študenti, ktorí preferujú alebo vyžadujú online formu konzultácie, sú tak nútení riešiť koordináciu platformy a odkazu na stretnutie mimo systému, čo zbytočne komplikuje celý proces.

²Diplomky FIT – Dostupné na: <https://www.diplomkyfit.cz/>



Obr. 2.2: Editačný mód systému Wiki@Merlin s ukážkou rezervácie konzultačného slotu prostredníctvom priamej editácie textového dokumentu.

2.1.3 Google Kalendár

Okrem špecifických fakultných systémov sa na organizáciu stretnutí využívajú aj globálne komerčné platformy. Jedným z nich je Google Kalendár³. Tento nástroj vyniká technickým spracovaním a vysokou spoľahlivosťou, a tak môže plne vyhovovať používateľom, ktorí preferujú centralizáciu a chcú mať všetky svoje osobné, pracovné aj školské udalosti na jednom mieste.

Hlavnou nevýhodou tohto riešenia v kontexte konzultácií na fakulte je však jeho príliš všeobecné zameranie, ktoré nedokáže vždy plne pokryť špecifické požiadavky vyučujúcich a študentov.

Google Kalendár neumožňuje logické členenie na miestnosti, bloky a sloty. Základné zdieľanie kalendára (kto môže vidieť termíny) je v Google Kalendári bezplatné. Obmedzenie prístupu ku konzultáciám výhradne pre konkrétne e-mailové adresy či vybrané domény – teda overenie, kto môže vykonať rezerváciu – je síce technicky možné, avšak táto funkcia je dostupná len v platených plánoch [11]. V bezplatnej verzii teda rezerváciu môže vykonať ktokoľvek, kto disponuje odkazom, čo otvára priestor pre fiktívne rezervácie a blokovanie termínov neoprávnenými osobami.

Ďalšou nevýhodou pre študentov je spôsob zobrazovania obsadenosti. Na rezervačnej stránke Google Kalendára sú zobrazené iba voľné termíny – obsadené sloty nie sú viditeľné. Keďže študenti svoje konzultácie často rušia alebo presúvajú, je pre nich v praxi oveľa výhodnejšie vidieť celý blok vrátane už obsadených slotov. Takýto prehľad im umožňuje sledovať aj plné termíny a prípadne si zapnúť upozornenia pre prípad, že by sa nimi preferovaný slot uvoľnil.

³Google Kalendár – online kalendárová služba od spoločnosti Google. Dostupné na: <https://calendar.google.com>

Diplomky FIT

Konzultace Domů > Konzultace > Přehled

Přehled konzultací - Vítězslav Beran

Změny v registraci na konzultace je možné provádět nejpozději 24 hodin před konzultací.

Čas konzultace	Student	Akce
24. 4. 2026		
11.5.2026		
12:00 - 12:20		Odhlásit
12:20 - 12:40		Odhlásit
12:40 - 13:00		Odhlásit
13:00 - 13:20		Odhlásit
13:20 - 13:40		Odhlásit

Obr. 2.3: Rozhraní systému Diplomky FIT so zoznamom dostupných termínov konzultácií. Ako príklad je zobrazený reálny rezervačný formulár konzultačných hodín doc. Ing. Vítězslava Berana, Ph.D.

Kombinácia uvedených obmedzení robí z Google Kalendára nástroj, ktorý síce exceluje ako univerzálne riešenie, avšak pre účely štruktúrovanej správy konzultácií na fakulte poskytuje len nedostatočnú funkčnú podporu.

2.1.4 Microsoft Excel

Ďalším príkladom globálneho komerčného riešenia, ktoré sa v praxi využíva na správu konzultačných hodín, je tabuľkový editor Microsoft Excel⁴. Podobne ako pri Wiki@Merlin, aj toto riešenie sa spolieha na disciplínu a pozornosť samotných používateľov, keďže neposkytuje žiadnu ochranu jednotlivých záznamov. Každý používateľ s oprávnením na úpravu súboru môže omylom alebo zámerne prepísať či vymazať záznamy iných študentov, čo vystavuje celý harmonogram konzultácií riziku neúmyselného poškodenia alebo straty dát.

Z hľadiska prístupu platí, že zdieľaný excelový súbor býva typicky uložený na platforme Microsoft OneDrive alebo SharePoint, ktorých prístup je vo fakultnom prostredí viazaný na školský účet.

Ovládanie mobilnej aplikácie Microsoft Excel pri práci s rezervačnými tabuľkami je menej pohodlné. Keďže tabuľky bývajú rozsiahle v oboch osiach, používateľ je nútený posúvať zobrazenie horizontálne aj vertikálne, aby získal úplný prehľad o dostupných termínoch. Orientácia v takomto prostredí na malom displeji smartfónu je nepohodlná a zvyšuje riziko neúmyselných chýb pri editácii.

Čo sa týka notifikácií, Microsoft Excel v kombinácii s OneDrive alebo SharePoint umožňuje prijímať e-mailové upozornenia pri zmene súboru prostredníctvom funkcie *Alert Me* alebo automatizovaných tokov v nástroji Power Automate [16]. Tieto notifikácie sú však

⁴Microsoft Excel – tabuľkový editor od spoločnosti Microsoft. Dostupné na: <https://www.microsoft.com/microsoft-365/excel>

Obr. 2.4: Rezervačná stránka Google Kalendára zobrazujúca výlučne voľné termíny – obsadené sloty nie sú pre návštevníka viditeľné. Ako príklad je zobrazený reálny rezervačný formulár konzultačných hodín Ing. Tomáša Mileta, Ph.D.

Tabuľka 2.1: Porovnanie existujúcich rezervačných systémov.

Vlastnosť	Wiki@Merlin	Diplomky FIT	Google Kalendár	Excel
Ochrana rezervácií	×	✓	✓	×
Mobilná aplikácia	×	×	✓	✓
Kontrola prístupu	✓	✓	○	✓
Viditeľnosť obsadených slotov	✓	✓	×	✓
E-mailové notifikácie	×	✓	✓	●
Online stretnutia	✓	×	✓	✓

° Overenie e-mailu pred rezerváciou (obmedzenie prístupu) je dostupné len v platených plánoch. Základné zdieľanie kalendára je bezplatné.

• Notifikácia len na úrovni celého súboru, nie jednotlivých zmien.

dostupné len na úrovni celého súboru – systém informuje o tom, že bol súbor zmenený, nie o tom, ktorý konkrétny blok bol upravený.

Napriek týmto obmedzeniam Excel ako rezervačný nástroj funguje – je široko dostupný a pre jednoduché scenáre postačuje. Ide však o všeobecný nástroj, ktorý nebol navrhnutý pre účely štruktúrovanej správy konzultácií, a preto chýbajú funkcie ako logické členenie na miestnosti a bloky alebo automatická ochrana rezervácií.

2.1.5 Zhodnotenie a motivácia pre vlastné riešenie

Tabuľka 2.1 sumarizuje kľúčové vlastnosti analyzovaných systémov. Z porovnania vyplýva, že hoci každý z nich pokrýva určitú podmnožinu požadovanej funkcionality, žiadny nespĺňa všetky potreby súčasne. Systém Wiki@Merlin je technologicky zastaraný a nezabezpečuje ochranu dát pred nežiaducou modifikáciou. Systém Diplomky FIT síce rieši integritu dát, ale chýba mu mobilná podpora a možnosť online konzultácií. Globálne riešenie Google Kalendár zasa nezohľadňuje špecifické potreby fakulty a v bezplatnej verzii neposkytuje dostatočnú kontrolu prístupu. Microsoft Excel síce ponúka zdieľanú editáciu, avšak ako univerzálny tabuľkový editor postráda ochranu jednotlivých rezervácií a neposkytuje štruk-

Zapíšte sa do zvoleného boxu, prosím, neupravujte konzultácie ostatných ani nové sloty nepridávejte. Konzultácie jsou defaultně prezenční v Q202, není-li uvedeno jinak, pokud chcete online. V případě, že potřebujete více času, zarezervujte si klidně dva sloty Invite link na MS Teams: https://teams.microsoft.com/j/team/19%3aVilclR1hPT2wGCoX4099SYvRdhvKNazL5j8VZdTqR_Q1%40thread.tacy2/conversations?groupId=90df2afc-43a9-409b-a141-44656bc2a411&tenantId=c63ce72									
ZIMNÍ SEMESTR					LETNÍ SEMESTR				
16.9.2025	jméno	login	komentář	moje poznámky	10.2.2026	jméno	login	komentář	moje poznámky
15:00					15:00			DP	
15:20			BP		15:20			BP	
15:40			DP - online		15:40				
16:00			BP		16:00				
16:20					16:20				
16:40			BP		16:40				
17:00					17:00				
23.9.2025	jméno	login	komentář		17.2.2026	jméno	login	komentář	
15:00			BP - online		13:40				Pozor, výjimečně posunut čas začátku konzultaci.
15:20			BP - online		14:00				
15:40			BP - online		14:20				
16:00			BP		14:40				
16:20			DP		15:00				
16:40			BP		15:20				
17:00					15:40				

Obr. 2.5: Rezervační stránka Excel. Ako příklad je zobrazený excel dokument Ing. Michala Vlnasa.

túrovaný prehľad vhodný pre správu konzultačných hodín. Tieto nedostatky tvoria priamu motiváciu pre vývoj vlastnej aplikácie.

2.2 Špecifikácia požiadaviek na systém

Požiadavky na systém vychádzajú z analýzy existujúcich riešení a definovaných cieľov práce. Rozdeľujú sa na funkčné – čo musí systém vedieť robiť – a nefunkčné – akými vlastnosťami musí disponovať.

2.2.1 Funkčné požiadavky

Používateľské roly nie sú viazané na profesijné označenie, ale na vzťah k miestnosti. Jeden používateľ tak môže byť v jednej miestnosti Majiteľom a v inej Návštevníkom. To znamená, že rovnaký používateľ – napríklad doktorand, ktorý vedie vlastné konzultácie za cvičenia, ale zároveň chodí na konzultácie k svojmu školiteľovi – by v systéme nemohol byť súčasne učiteľom aj študentom, keby sme roly nazvali „Teacher“ a „Student“. Preto sú roly definované výlučne vzťahom ku konkrétnej miestnosti: v jednej miestnosti vystupuje ako jej Majiteľ (zodpovedá učiteľovi), v inej len ako Návštevník (zodpovedá študentovi).

- **Autentifikácia:** Registrácia a prihlásenie prebiehajú cez e-mailovú adresu a jednorazový OTP kód. Nový používateľ nemá prístup k žiadnym dátam, kým ho Majiteľ nepridá do svojej miestnosti.
- **Správa miestností (Majiteľ):** Vytváranie, úprava a mazanie konzultačných miestností.
- **Správa blokov a slotov (Majiteľ):** Vytváranie, úprava a mazanie časových blokov a slotov v rámci nich.

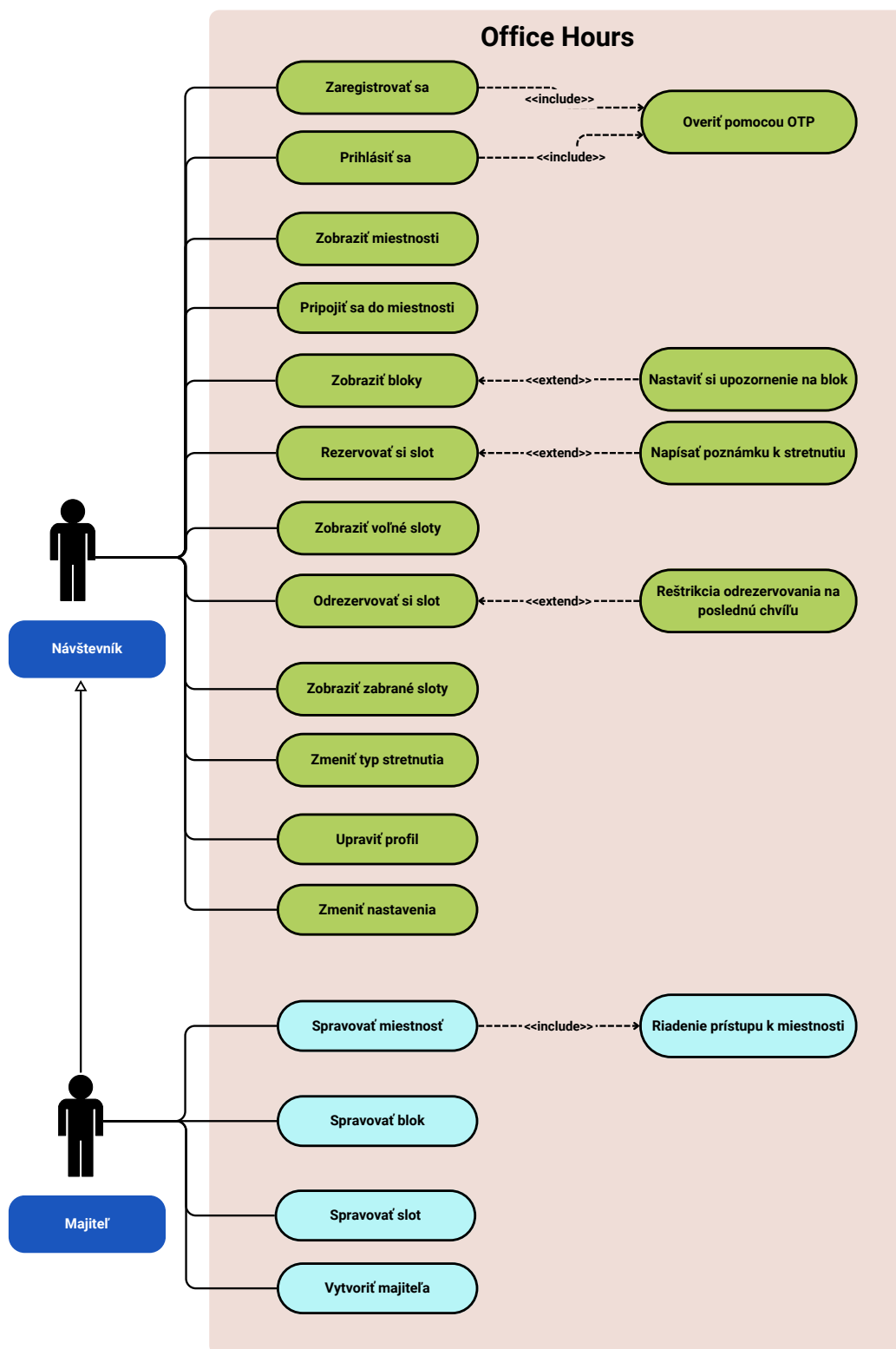
- **Riadenie prístupu (Majiteľ):** Správa zoznamu povolených e-mailových adries pre každú miestnosť.
- **Rezervácia (Návštevník):** Prezeranie dostupných miestností, blokov a slotov a rezervovanie voľného slotu s možnosťou pridania textovej poznámky popisujúcej dôvod stretnutia.
- **Zrušenie rezervácie (Návštevník):** Návštevník môže zrušiť vlastnú rezerváciu. Ak ju zruší neskoro, systém ho na túto skutočnosť upozorní.
- **Nastavenia a GDPR:** Používateľ si môže prispôbiť vizuálny motív a skryť svoje meno voči ostatným používateľom.
- **E-mailové notifikácie:** Automatické upozornenia obom stranám pri zmenách, pokiaľ to používateľ povolí v nastaveniach.
- **Prehľad pripojených používateľov (Majiteľ):** Majiteľ má k dispozícii prehľad o tom, ktorí návštevníci sú v danej miestnosti pripojení.

2.2.2 Nefunkčné požiadavky

- **Multiplatformová podpora:** Aplikácia musí byť plne funkčná na Androide aj iOS z jedinej kódovej základne (Flutter).
- **Bezpečnosť:** Prístup k dátam je povolený výlučne autentifikovaným používateľom. Spojenie so serverom musí byť udržiavané bezpečným mechanizmom bez nutnosti opakovane zadávať prihlasovacie údaje, ak si to používateľ praje.
- **Odozva (Optimistic UI):** Pri bežných interakciách (rezervácia, zrušenie) aplikácia reaguje vizuálne okamžite a potvrdenie prebehne asynchrónne na pozadí. Ak zmena nebude úspešná, aplikácia sa vráti do pôvodného stavu. Blokujúce načítavacie obrázky sú vyhradené len pre nevyhnutné operácie ako napríklad počiatočné stiahnutie dát.
- **Používateľské rozhranie:** UI musí rešpektovať moderné dizajnové štandardy a umožňovať intuitívnu navigáciu.

2.2.3 Modelovanie prípadov použitia (Use Cases)

Na základe definovaných požiadaviek bol zostavený diagram prípadov použitia (Obrázok 2.6), ktorý vizualizuje interakcie aktérov so systémom. Vzťah medzi Majiteľom a Návštevníkom je definovaný generalizáciou – Majiteľ dedí všetky oprávnenia Návštevníka a navyše disponuje právami na správu rezervačného priestoru. Každý používateľ musí prejsť autentifikáciou. Relácia «**include**» definuje, že operácie „Zaregistrovať sa“ aj „Prihlásiť sa“ povinne zahŕňajú krok „Overiť pomocou OTP“. Návštevník má k dispozícii funkcie prezerania miestností, blokov a slotov, pričom relácia «**extend**» umožňuje voliteľné správanie – nastavenie upozornenia na blok, pridanie poznámky k rezervácii či upozornenie pri rušení rezervácie na poslednú chvíľu. Z pohľadu Majiteľa diagram definuje administrátorské funkcie: „Spravovať miestnosť“, „Spravovať blok“ a „Spravovať slot“, pričom „Riadenie prístupu k miestnosti“ (zahrnuté cez «**include**») umožňuje obmedziť prístup na konkrétne e-mailové adresy alebo celé domény.



Obr. 2.6: Diagram prípadov použitia aplikácie Office Hours zobrazujúci interakcie dvoch hlavných aktérov – Návštevníka a Majiteľa.

Kapitola 3

Použité technológie, nástroje a architektonický vzor

Táto kapitola analyzuje zvolené nástroje, technológie a architektonický vzor využité počas celého životného cyklu mobilnej aplikácie Office Hours – od vizuálneho návrhu a prototypovania používateľského rozhrania, cez implementáciu klientskej časti až po testovanie komunikácie s backendovými službami.

Nástroje použité pri vývoji a návrhu

- **Postman:** Platforma na testovanie API, ktorá bola použitá na overovanie komunikácie s backendom, testovanie autentifikácie pomocou JWT (JSON Web Token) tokenov a kontrolu HTTP status kódov¹.
- **Figma:** Cloudový vektorový grafický editor využitý na tvorbu kompletných vizualizácií všetkých obrazoviek a interaktívnych prototypov používateľského rozhrania².
- **Canva:** Online grafický editor použitý na tvorbu Use Case diagramu a diagramu toku autentifikácie³.
- **Coolers.co:** Webový generátor farebných schém s integrovaným Contrast Checkerom⁴.

3.1 Použité technológie

Nasledujúca sekcia predstavuje primárne technológie, programovacie jazyky, zvolenú architektúru a špecifické knižnice, ktoré tvoria funkčný základ mobilnej aplikácie.

3.1.1 Framework Flutter a jazyk Dart

Na vývoj mobilnej aplikácie bol použitý **Flutter** vo verzii 3.32.7 – open-source UI framework vyvinutý spoločnosťou Google [10]. Výber Flutteru oproti alternatívnym multiplatformovým riešeniam bol podmienený niekoľkými technickými faktormi.

¹<https://www.postman.com>

²<https://www.figma.com>

³<https://www.canva.com>

⁴<https://coolers.co>

Na rozdiel od React Native, ktorý pri tvorbe UI závisí od natívnych komponentov platformy a komunikuje s nimi cez JavaScript vrstvu, čo môže spôsobovať výkonnostnú réžiu a vizuálnu nekonzistentnosť medzi platformami (hoci nová architektúra React Native JSI-/Fabric, dostupná od verzie 0.68, tieto problémy výrazne redukuje [15]), flutter vykresľuje všetky grafické prvky prostredníctvom vlastného enginu Impeller (nahradza pôvodný engine Skia) nezávisle od operačného systému. Tento prístup zaručuje identický vzhľad a správanie aplikácie na systémoch Android aj iOS, pričom v produkčnom zostavení Flutter kompiluje kód priamo do natívnych inštrukcií procesora [8].

V porovnaní s Kotlin Multiplatform (KMP) je Flutter vhodnejší najmä pri rýchlom prototypovaní a vývoji MVP (Minimum Viable Product). KMP síce umožňuje zdieľanie biznis logiky, avšak používateľské rozhranie sa stále implementuje natívne pre každú platformu osobitne. V praxi to znamená, že Android a iOS môžu používať rovnakú logiku, ale používateľské rozhranie sa stále implementuje natívne, (napríklad pomocou Jetpack Compose na Androide a SwiftUI na iOS). Takýto prístup je výhodný pri aplikáciách, kde je prioritou natívny vzhľad a hlboká integrácia s platformou, no natívne rozhranie pre iOS si zároveň vyžaduje dodatočnú znalosť jazyka Swift/SwiftUI. Flutter naproti tomu pokrýva v jedinom projekte a jedinom jazyku (Dart) ako biznis logiku, tak aj celé používateľské rozhranie [13].

Základným stavebným prvkom vo Flutteri je tzv. *Widget*. Vo Flutteri je widgetom úplne všetko – od textu a tlačidla až po celkové rozloženie obrazovky (layout). Aplikácia vzniká skladaním týchto widgetov do hierarchického stromu (Widget Tree) [7].

Aplikácie vo Flutteri sú programované v jazyku **Dart**, ktorý je rovnako vyvíjaný spoločnosťou Google. V tomto projekte bola použitá verzia Dart SDK 3.8.1. Ide o moderný, objektovo orientovaný jazyk so silnou typovou kontrolou. Jeho zásadnou výhodou pre mobilný vývoj je podpora dvoch typov kompilácie [4]:

- **JIT (Just-In-Time):** Využíva sa počas vývoja. Umožňuje funkciu Hot Reload, keď sa zmeny v kóde prejaví v bežiacей aplikácii za zlomok sekundy bez nutnosti rekompilácie.
- **AOT (Ahead-Of-Time):** Využíva sa pri tvorbe produkčného zostavenia (release build). Kód je skompilovaný priamo do inštrukcií procesora, čo zabezpečuje rýchly štart aplikácie a vysoký výkon na koncovom zariadení.

Jazyk Dart taktiež podporuje bezpečnosť voči nulovým hodnotám, čím sa priamo na úrovni kompilátora predchádza jednej z najčastejších chýb – chybám spôsobeným prístupom k nulovým hodnotám [6].

3.1.2 Použité knižnice (Packages)

Projekt využíva balíčky z oficiálneho repozitára *pub.dev*⁵. Medzi najdôležitejšie knižnice integrované do aplikácie Office Hours patria:

- **http:** Slúži na tvorbu sieťových požiadaviek. Táto knižnica bola použitá na komunikáciu s dedikovaným REST API⁶ (odosielanie a prijímanie JSON dát pomocou metód GET, POST a DELETE).
- **provider:** Knižnica pre správu stavu aplikácie vo Flutteri, využitá na implementáciu architektúry MVVM. Konkrétny spôsob prepojenia ViewModelu s prezentačnou vrstvou je popísaný v sekcii 3.2.1.

⁵<https://pub.dev>

⁶REST API, s ktorým aplikácia komunikuje, je dostupné na: <https://office-hours.fit.vutbr.cz/api/>

- **get_it**: Tento balíček slúži na centrálnu registráciu a prístup k službám, akými sú napríklad triedy **ApiService** alebo **NavigationService**, čím sa zabezpečuje čisté oddelenie zodpovedností v kóde [3].
- **shared_preferences**: Umožňuje perzistentné ukladanie jednoduchých dátových typov (napr. používateľské preferencie, zvolená téma) do nešifrovaného lokálneho úložiska zariadenia.
- **flutter_secure_storage**: Zabezpečuje ukladanie citlivých údajov do šifrovaného systémového úložiska zariadenia [18]. V aplikácii je použitá výlučne na ukladanie JWT tokenu. Konkrétna implementácia je popísaná v sekcii 5.4.2.

Pre vizuálne a pomocné účely boli ďalej využité nasledujúce balíčky:

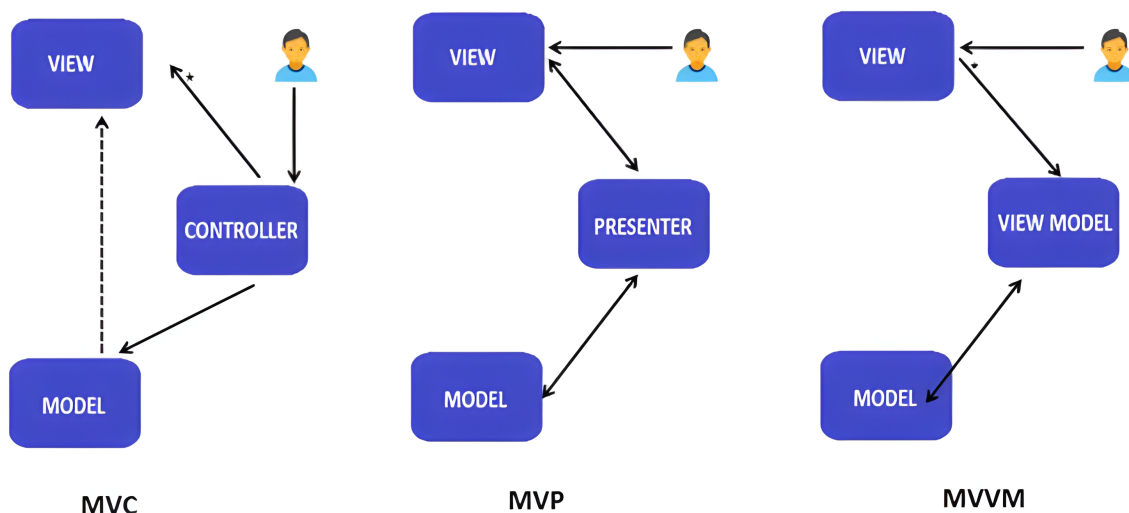
- **flutter_svg** s ikonami *Heroicons*⁷: Integrácia sady ikon vo formáte SVG.
- **fluttertoast** a **overlay_kit**: Zobrazovanie notifikačných správ (Toast notifications).
- **flutter_spinkit**: Animované indikátory načítavania.
- **flutter_native_splash** a **flutter_launcher_icons**: Konfigurácia ikony aplikácie a úvodnej splash obrazovky.
- **intl**: Formátovanie dátumov a časov.
- **dropdown_search**: Vyhľadávateľné rozbaľovacie zoznamy (autocomplete).
- **sticky_headers**: Knižnica zabezpečujúca pripnutie hlavičiek sekcií pri rolovaní zoznamom.
- **connectivity_plus**: Detekcia stavu sieťového pripojenia.
- **jwt_decoder**: Overenie expirácie JWT tokenu.
- **url_launcher**: Otváranie externých odkazov v systémovom prehliadači.
- **syncfusion_flutter_datepicker**: Komponent kalendára s výberom dátumu.
- **figma_squircle**: Tvar, ktorý je prechodom medzi štvorcom (square) a kruhom (circle).

3.2 Architektonický vzor MVVM

Bez jasne definovanej architektúry sa kód mobilnej aplikácie rýchlo stáva neudržateľným a neprehľadným. Z tohto dôvodu bol v projekte Office Hours implementovaný architektonický návrhový vzor **MVVM (Model-View-ViewModel)**.

Na pochopenie prínosu MVVM je užitočné porovnať ho so starším vzorom **MVC (Model-View-Controller)**, z ktorého vychádza. Porovnanie komunikácie medzi jednotlivými vrstvami architektúr MVC, MVP a MVVM je znázornené na obrázku 3.1. V MVC sprostredkúva interakciu medzi dátami a rozhraním komponent Controller. Jeho zásadným problémom je *smer závislosti*: Controller priamo ovláda View, drží referencie na konkrétne

⁷<https://heroicons.com/>



Obr. 3.1: Porovnanie architektonických vzorov MVC, MVP a MVVM – znázornenie komunikácie medzi používateľom, View, Modelom a riadiacimi komponentmi. Zdroj: [20]

UI prvky a aktívne do nich tlačí zmeny (napríklad `controller.hideButton()` alebo `controller.updateLabel()`). Výsledkom je pevná väzba – akákoľvek zmena v UI si vyžaduje zásah aj do Controllera, čo kód robí krehkým a ťažko testovateľným [1].

Ďalším relevantným vzorom je **MVP (Model-View-Presenter)**, ktorý priamo predchádza MVVM a rieši niektoré nedostatky MVC. Presenter úplne oddeľuje vrstvu Model od vrstvy View, View a Presenter komunikujú prostredníctvom explicitného rozhrania (interface), čo výrazne uľahčuje testovanie. Nevýhodou MVP je však to, že s rastom projektu sa Presenter môže stať rozsiahlou triedou s množstvom metód, čo komplikuje jeho pochopenie a údržbu [1].

Pri MVVM je ViewModel úplne nezávislý od View [1]. ViewModel namiesto priamych volaní na View vystavuje pozorovateľný stav – napríklad pomocou **StateFlow** v Kotlinu alebo **ObservableObject** v SwiftUI – a View sa naň prihlási ako pozorovateľ.

Tento vzor striktne rozdeľuje aplikačný kód do troch hlavných vrstiev:

- **Model (Dátová vrstva):** Obsahuje definície dátových štruktúr a služby zodpovedné za komunikáciu s externým API. Táto vrstva nemá žiadnu vedomosť o používateľskom rozhraní.
- **View (Prezentačná vrstva):** Zodpovedá výlučne za vizualizáciu dát. Vo Flutteri ide o sadu widgetov, ktoré zobrazujú stav z ViewModelu a predávajú mu informácie o používateľských interakciách, ako sú kliknutia alebo gestá.
- **ViewModel (Logická medzivrstva):** Služi ako most medzi Modelom a View. Udržiava aktuálny stav obrazovky, spracováva vstupy a transformuje dáta získané z Modelu do formátu zobraziteľného v UI.

3.2.1 Reaktívne prepojenie View a ViewModelu

V projekte Office Hours je reaktívne prepojenie medzi View a ViewModelom zabezpečené prostredníctvom balíčka **provider** [19] v kombinácii s triedou **ChangeNotifier**. Toto riešenie je postavené na návrhovom vzore **Pozorovateľ** (Observer pattern).

Princíp fungovania je nasledovný: keď ViewModel vykoná asynchrónnu operáciu (napríklad odošle požiadavku na server), zmení svoj lokálny stav a zavolá metódu **notifyListeners()**. Toto volanie upozorní framework Flutter, aby prekreslil *výlučne* tú časť používateľského rozhrania, ktorá počúva na zmeny daného ViewModelu. Vďaka tomuto selektívnemu prístupu nedochádza k zbytočnému prekresľovaniu celej obrazovky, čo šetrí systémové prostriedky a prispieva k plynulosti aplikácie.

Kapitola 4

Návrh používateľského rozhrania a štruktúry aplikácie

Táto kapitola opisuje vizuálny návrh aplikácie Office Hours: od prvých ručných náčrtov cez prototyp až po návrh štruktúry projektu a možnosti budúceho rozšírenia.

4.1 Návrh používateľského rozhrania

Cieľom návrhu používateľského rozhrania bolo vytvoriť moderné a predovšetkým intuitívne prostredie, ktoré umožňuje používateľovi rýchlo a bezchybne vykonať požadovanú akciu.

4.1.1 Od počiatočných náčrtov k prototypu vo Figma

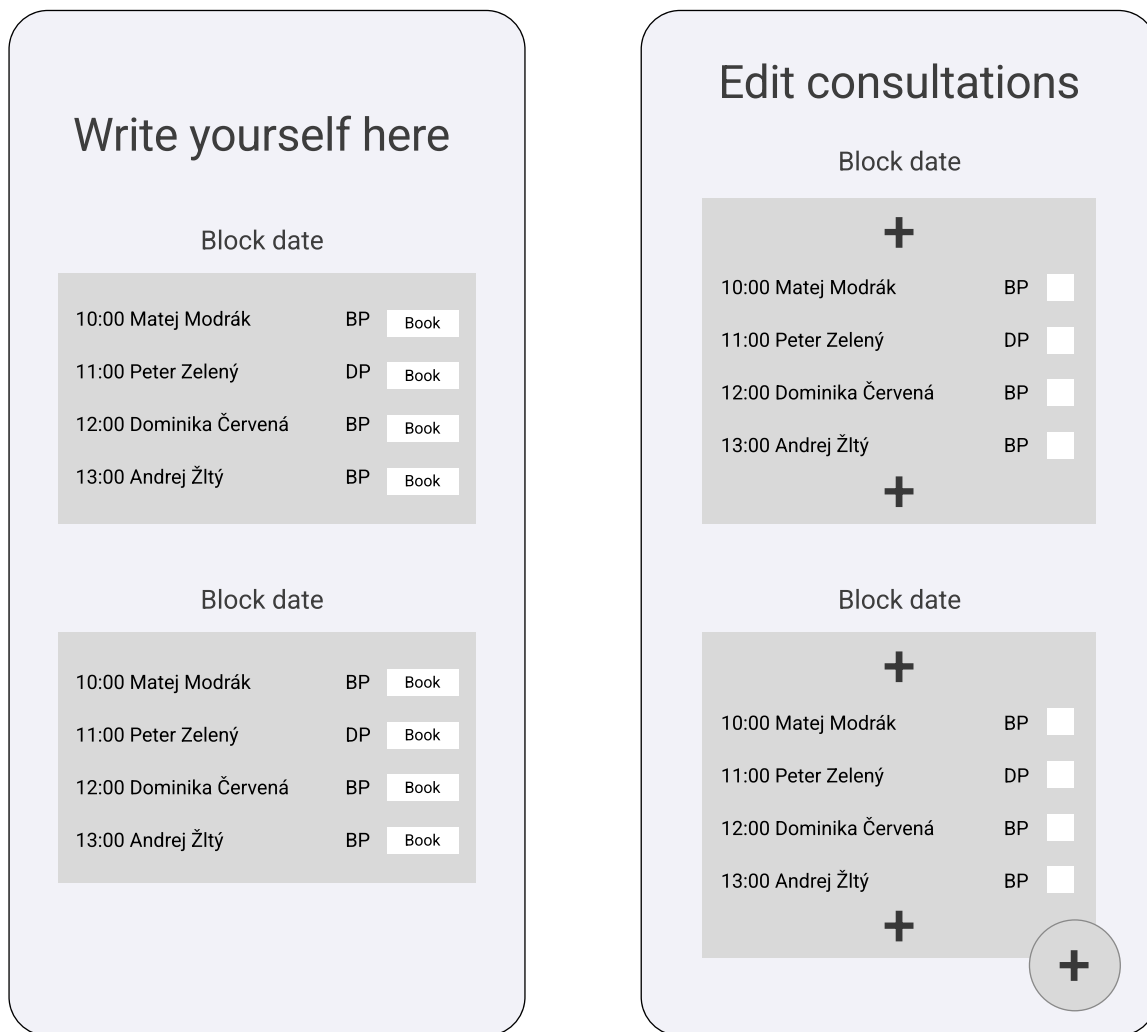
Proces návrhu používateľského rozhrania začal sériou iteratívnych konzultácií s vedúcim bakalárskej práce. Prvotná vízia rozmiestnenia kľúčových prvkov – ako sú zoznamy miestností, časové bloky a rezervačné dialógy – bola vytvorená vo forme jednoduchých ručných náčrtov (tzv. wireframes). Tento prístup umožnil rýchlu a flexibilnú validáciu základnej informačnej architektúry.

Následne bol tento overený koncept pretransformovaný do interaktívneho prototypu v grafickom editore Figma. Pri tomto návrhu bol striktne dodržiavaný prístup „Mobile-first“, čo znamená, že aplikácia bola primárne navrhovaná pre malé dotykové obrazovky smartfónov [21]. Tento prístup zaručil, že kľúčové ovládacie prvky (napríklad potvrdzovacie tlačidlá alebo navigácia) sú umiestnené v tzv. zóne pohodlného dosahu palca, zatiaľ čo menej dôležité informácie alebo nadpisy sa nachádzajú v hornej časti obrazovky.

4.1.2 Princípy použiteľnosti pre user interface (UI) dizajn

Návrh aplikácie Office Hours sa riadil heuristikami použiteľnosti (Usability Heuristics) podľa Jakoba Nielsena [17].

- **Viditeľnosť stavu systému (Visibility of system status):** Systém by mal používateľa vždy vhodne a včas informovať o tom, čo sa deje. Táto heuristika je implementovaná vizuálnou spätnou väzbou: po odoslaní požiadavky na server sa tlačidlo okamžite zmení na indikátor načítavania, čím sa zabráni viacnásobnému odoslaniu formulára. Rovnako po dokončení každej operácie (napríklad rezervácií slotu) systém zobrazí potvrdzujúcu Toast notifikáciu.



Obr. 4.1: Wireframy znázorňujúce domovskú obrazovku Návštevníka (naľavo) a Majiteľa (napravo).

- **Zhoda medzi systémom a skutočným svetom:** Systém by sa mal inšpirovať správaním z reálneho sveta. V aplikácii sa to prejavuje okrem iného prirodzenou fyzikou scrollovania (tzv. rubber-band effect) – keď používateľ dosiahne koniec zoznamu slotov, obrazovka sa prirodzene natiahne a odpruží späť, čím signalizuje používateľovi koniec obsahu.
- **Kontrola a sloboda používateľa (User control and freedom):** Používatelia musia mať vždy k dispozícii jasne viditeľné tlačidlo pre núdzový odchod z nežiaduceho stavu. Každá vnorená obrazovka v aplikácii (napríklad detail miestnosti) preto obsahuje výraznú šípku na krok späť v ľavom hornom rohu. Na domovskej obrazovke túto rolu plní vysúvacie „hamburger“ menu. Pri deštruktívnych akciách (napríklad zmazanie miestnosti) systém vyžaduje explicitné potvrdenie dialógovým oknom.
- **Konzistencia a štandardy:** Konzistencia umožňuje používateľovi vytvoriť si stabilný mentálny model aplikácie. V Office Hours je uplatňovaná jednotnou terminológiou, vizuálnou štandardizáciou a 4pt grid systémom, ktorého deliteľnosť hodnotami

2, 4 a 8 umožňuje prirodzené škálovanie komponentov [2]. Dizajnový systém je striktne obmedzený na 4 veľkosti písma, 2 hrúbky a úzky set farieb.

- **Prevenencia chýb (Error prevention):** Aplikácia je navrhnutá tak, aby bolo ťažké urobiť chybu. Napríklad textové polia používajú špecifické typy klávesníc (napríklad email pre e-mailové adresy) a validácia prebieha ešte pred odoslaním na server.
- **Rozpoznanie, nie vybavovanie si (Recognition rather than recall):** Zataženie pamäte používateľa sa minimalizuje tým, že sú všetky dôležité akcie a informácie viditeľné. Namiesto toho, aby si používateľ pamätal identifikátory slotov, aplikácia zobrazuje všetky termíny farebne odlíšené (biele = voľné, červené = cudzie, modré = vlastné). Autocomplete pole na obrazovke pripojenia do miestnosti dynamicky dopĺňa názvy ešte počas písania, čím používateľa zbavuje nutnosti pamätať si presný názov miestnosti.
- **Flexibilita a efektívnosť používania (Flexibility and efficiency of use):** Systém by mal vyhovovať neskúseným aj pokročilým používateľom. V Office Hours je to implementované viacerými spôsobmi: Majiteľ môže vytvoriť nový blok rýchlym tlačidlom s ikonkou plus priamo na hlavnej obrazovke (bez nutnosti prechádzať menu). Pri vytváraní bloku podporuje výber viacerých dátov naraz (režim „range“ alebo „multiple“). Prepínač medzi rolami (Owner/Visitor) umožňuje rýchlu zmenu kontextu bez prihlasovania a odhlasovania. Pokročilí používatelia môžu využívať funkcie ako kopírovanie celých blokov na iný dátum.

4.1.3 Aplikácia dizajnových princípov a vizuálna identita

Farebná paleta využíva neutrálne pozadie, ktoré necháva vyniknúť kľúčové údaje a interaktívne prvky. Primárnou farbou je sýta modrá, používaná výhradne na interaktívne komponenty (tlačidlá, aktívne záložky), aby si používateľ rýchlo vytvoril mentálny model klikateľných prvkov. Pre deštruktívne akcie sa používa červená farba.

Typografia je navrhnutá tak, aby vytvárala jasnú vizuálnu hierarchiu primárne cez veľkosť a hrúbku písma, nie zmenou farieb. Nadpisy (napríklad názvy miestností) sú výrazné a ľahko skenovateľné, doplnkové informácie sú zobrazené menším písmom a so zníženou nepriehľadnosťou, čo prirodzene vedie oko od najdôležitejších prvkov k detailom.

Výrazným vizuálnym prvkom sú tvary typu squircle, teda zaoblenejšie superelipsy známe z iOS dizajnu. Tieto tvary pôsobia jemnejšie než klasické obdĺžniky s jednoduchým border-radius, čím rozhraniu dodávajú modernejší a priateľskejší charakter.

4.2 Návrh jednotlivých častí aplikácie

Táto sekcia postupne prechádza návrhom obrazoviek aplikácie Office Hours – od autentifikácie cez pohľad Návštevníka až po rozhranie Majiteľa miestnosti – a opisuje rozhodnutia, ktoré formovali výsledný používateľský tok.

4.2.1 Úvodná obrazovka a autentifikácia

Prvým vizuálnym dojmom aplikácie je splash obrazovka (úvodná dočasná obrazovka, ktorá sa zobrazí pri spustení aplikácie) s logom Office Hours. Počas jej zobrazovania prebehne inicializácia aplikácie.

Po splash obrazovke nasleduje prihlasovacia obrazovka, kde používateľ zadá svoju e-mailovú adresu. Kľúčovým dizajnovým rozhodnutím je, že aplikácia sama rozhodne, čo nasleduje. Používateľ sa tak nemusí rozhodovať medzi možnosťami prihlásenia a registrácie. Ak e-mail nie je evidovaný v systéme, aplikácia ho plynulo presmeruje na registračnú obrazovku s predvyplneným e-mailom, kde doplní meno, priezvisko a účel návštevy. Ak účet už existuje, pokračuje priamo na overenie.

Toto riešenie prináša UX výhodu v podobe zjednodušeného vstupného toku, no zároveň so sebou nesie bezpečnostný kompromis. Rozdielna reakcia aplikácie na registrovaný a neregistrovaný e-mail umožňuje tzv. enumeráciu používateľov – útočník môže systematickým zadávaním e-mailových adries zistiť, ktoré z nich sú v systéme registrované. V kontexte tejto aplikácie, určenej primárne pre akademické prostredie s obmedzeným okruhom používateľov, bolo toto riziko vedome akceptované v prospech jednoduchosti používateľského zážitku.

V oboch prípadoch nasleduje obrazovka overenia pomocou jednorazového kódu (OTP), ktorý je doručený na zadaný e-mail. Tento krok zabráňuje zneužitiu cudzích e-mailových adries a neoprávnenému zaberaniu miesta v databáze.

4.2.2 Pohľad Návštevníka

Úvodná obrazovka a pripojenie do miestnosti

Po úspešnom prihlásení sa Návštevník, ktorý ešte nie je pripojený do žiadnej miestnosti, ocitne na úvodnej obrazovke. Tá mu zobrazí výzvu na pripojenie sa do prvej miestnosti spolu s priamym odkazom na obrazovku Join Room, čím sa eliminuje zbytočná navigácia.

Na obrazovke Join Room sa nachádza textové pole, ktoré počas písania automaticky dopĺňa názvy dostupných miestností (autocomplete). Používateľ vyberie požadovanú miestnosť zo zoznamu a potvrdí pripojenie.

Hlavná obrazovka s miestnosťami, blokmi a slotmi

Po pripojení do miestnosti sa Návštevník dostane na hlavnú obrazovku konzultácií. V hornej časti obrazovky sa nachádza prepínač miestností (room selector), ktorý umožňuje rýchle prepínanie medzi viacerými miestnosťami v prípade, že sa Návštevník konzultuje u viacerých vyučujúcich. Priamo pod ním je umiestnené odkazové tlačidlo smerujúce na externý link miestnosti. Napríklad na link pre online konzultácie alebo na osobnú stránku vyučujúceho.

Ak v miestnosti zatiaľ neboli vytvorené žiadne bloky, obrazovka zobrazí informatívny text. V opačnom prípade sú zobrazené bloky s ich slotmi. Hlavička každého bloku obsahuje dátum konania a počet dní zostávajúcich do začiatku bloku, čo Návštevníkovi pomáha pri plánovaní. Voliteľne si tu môže aktivovať notifikácie pre daný blok, aby bol upozornený napríklad v prípade uvoľnenia obsadeného slotu.

Samotné sloty sú farebne odlíšené podľa svojho stavu:

- **Biele:** Voľný slot označený ikonkou plus v krúžku, ktorý je možné rezervovať kliknutím na neho.
- **Červené:** Obsadený cudzí slot s textom a ikonkou v utlmenej farbe, ktorá na červenom pozadí vytvára vizuálny dojem neprístupnosti a signalizuje, že slot nie je možné rezervovať.
- **Modré:** Vlastný rezervovaný slot patriaci prihlásenému Návštevníkovi.

Pri kliknutí na voľný (biely) slot sa zobrazí vysúvací dialóg (*bottom sheet*), ktorý Návštevníkovi umožní rezerváciu potvrdiť a zároveň ju bližšie špecifikovať, ako je znázornené na obrázku 4.8. Dialóg obsahuje predvyplnené textové pole s dôvodom návštevy (*visit purpose*), ktorý si používateľ nastavil v nastaveniach, pričom ho môže pred odoslaním ľubovoľne upraviť. Ďalšou voliteľnou možnosťou je prepínač online/offline konzultácie. Tento prepínač je však dostupný len v prípade, ak Majiteľ daný blok explicitne neoznačil ako online – napríklad z dôvodu choroby alebo iných okolností vynucujúcich dištančnú formu. V takom prípade je online režim pevne nastavený samotným Majiteľom a Návštevník ho nemôže zmeniť. Ak Majiteľ online režim bloku neurčil, Návštevník si môže sám zvoliť, či preferuje konzultáciu prezenčne alebo online.

Na vlastnom (modrom) slote sa nachádza ikonka *X* umožňujúca zrušenie rezervácie a ikonka obrazovky signalizujúca, že konzultácia prebehne online. V tele slotu je zobrazený čas konania a dôvod návštevy (*visit purpose*).

Toto riešenie sa zámerne odlišuje od Google Kalendára analyzovaného v kapitole 2, kde sú obsadené sloty pre návštevníka úplne skryté. Viditeľnosť všetkých slotov umožňuje Návštevníkovi sledovať aj plné termíny a prípadne si aktivovať upozornenie pre prípad uvoľnenia preferovaného slotu.

Vývoj obrazovky Návštevníka prebiehal iteratívne na základe spätnej väzby. Prvý prototyp (obrázok 4.2) obsahoval dedikované tlačidlo *Book* pri každom slote a ikonky strážneho psa pre notifikácie, ktoré sa v testovaní ukázali ako príliš komplexné a pre používateľov nejasné. Ikonka online stavu bola taktiež nejednoznačná. Na základe konzultácií s vedúcim práce a spätnej väzby od spolužiakov bol vytvorený druhý prototyp (obrázok 4.3): tlačidlá *Book* boli odstránené v prospech priameho kliknutia na slot, ikonky strážneho psa boli nahradené štandardnou ikonkou zvončeka a ikonka online stavu bola nahradená ikonkou obrazovky, ktorá jednoznačnejšie implikuje online konzultáciu. Zároveň bol pridaný prepínač medzi rolami Návštevníka a Majiteľa.

4.2.3 Pohľad Majiteľa

Majiteľ má nad hlavnou obrazovkou s miestnosťami umiestnený toggle, ktorý mu umožňuje kedykoľvek prepnúť sa do režimu Návštevníka – napríklad v prípade, ak sám potrebuje navštíviť konzultácie iného vyučujúceho. Toto riešenie priamo vychádza zo špecifikácie požiadaviek uvedenej v kapitole 2, kde jeden používateľ môže v rôznych miestnostiach vystupovať v oboch rolách súčasne.

Priamo pod prepínačom miestností a odkazovým tlačidlom miestnosti sa nachádza tlačidlo s ikonkou plus, ktoré slúži na rýchle vytvorenie nového bloku bez nutnosti prechádzať cez menu. Vedľa neho je umiestnená ikonka troch bodiek (*3 dots*), ktorá po kliknutí zobrazí kontextové menu s možnosťami správy miestnosti: *Edit Room*, *Delete Room* a *Show Users Joined* – prehľad Návštevníkov pripojených do miestnosti.

Majiteľ má na hlavnej obrazovke oproti Návštevníkovi rozšírené možnosti správy jednotlivých slotov. Pri každom slote je dostupná ikonka histórie, ktorej kliknutím sa zobrazí história daného slotu. Majiteľ má zároveň možnosť meniť stav slotu medzi online a offline režimom, a to aj pri slotoch, ktoré sú už rezervované Návštevníkom. Práve v takom prípade sú obzvlášť užitočné e-mailové notifikácie – ak Majiteľ zmení napríklad prezenčnú konzultáciu na online z dôvodu choroby, systém automaticky upozorní dotknutého Návštevníka e-mailom, pokiaľ má notifikácie povolené v nastaveniach.

Podobne ako pri obrazovke Návštevníka, aj pohľad Majiteľa prešiel iteratívnym vývojom. V prvom prototypu (obrázok 4.4) chýbali tlačidlá na správu miestnosti a bloku, ikonka

plus na pridanie bloku bola umiestnená na spodku obrazovky a prekryvala sa s blokmi. Obsadené sloty boli rozlíšené len pomocou checkboxov bez farebného kódovania. Na základe spätnej väzby bol znova vytvorený druhý prototyp (obrázok 4.5): ikonka plus bola presunutá priamo pod odkaz na miestnosť, boli pridané farebné stavy slotov, tlačidlo *Edit Block* a menu pre správu miestnosti (*Edit Room*, *Delete Room*, *Show Users Joined*).

Vytvorenie a editácia miestnosti

Pri vytváraní miestnosti Majiteľ nastaví nasledujúce atribúty:

- **Title:** Názov miestnosti (napríklad *Martin's Room*).
- **Room description:** Popis miestnosti zobrazený Návštevníkom, napríklad číslo kancelárie na fakulte (*L224*).
- **Link:** Externý odkaz priradený k miestnosti, napríklad link na online konzultácie alebo osobná webová stránka vyučujúceho.
- **Cancellation hours:** Počet hodín pred začiatkom konzultácie, do ktorých môže Návštevník zrušiť svoju rezerváciu bez varovania. Pri neskoršom zrušení systém zobrazí upozornenie, že toto konanie plytvá časom ostatných študentov aj vyučujúceho.
- **Allowed e-mails list:** Správa prístupu k miestnosti. Majiteľ môže povoliť prístup konkrétnym e-mailovým adresám alebo celým doménam. Zadaním požadovaného textu a následným kliknutím na potvrdzujúcu ikonku vpravo pridá Majiteľ tento text do zoznamu povolených e-mailov, ktorý sa dá zobrazit nižšie.

Obrazovka editácie miestnosti umožňuje úpravu všetkých vyššie uvedených atribútov. Majiteľ má zároveň k dispozícii možnosť zobrazit zoznam Návštevníkov pripojených do miestnosti, zobrazit históriu slotov a natrvalo vymazať miestnosť.

Vytvorenie a editácia bloku

Pri vytváraní bloku Majiteľ najprv vyberie jeden alebo viacero dátumov. Výber podporuje dva režimy: výber jednotlivých dní (1-n nesúvisiacich dátumov) alebo výber súvislého časového rozsahu (*range*). Vďaka tejto funkcii môže Majiteľ jednou akciou vytvorit bloky na viacero dní naraz, čo výrazne znižuje administratívnu záťaž pri pravidelných konzultáciách. Následne nastaví ďalšie parametre bloku:

- **Block start time:** Čas začiatku bloku, teda prvého slotu v bloku, nastaviteľný pomocou časového scrollera.
- **Minutes per slot:** Trvanie jedného slotu v minútach, volený taktiež pomocou časového scrollera.
- **Slot count:** Počet slotov, ktoré budú vytvorené.
- **Block end time:** Na základe zadaného počtu slotov, ich trvania a času začatia prvého slotu sa automaticky vypočíta čas ukončenia bloku (*end time*).
- **Note:** Voliteľná textová poznámka pre Návštevníkov viditeľná v hlavičke bloku.
- **Is online:** Označenie bloku ako online konzultácie.

V obrazovke editácie bloku môže Majiteľ mazať jednotlivé sloty, prepínať blok a sloty medzi online a offline stavom, vymazať celý blok alebo ho nakopírovať na iný dátum. Pridanie nového slotu je možné tromi spôsobmi: kliknutím na tlačidlo plus pred prvým slotom v bloku, kliknutím na tlačidlo plus za posledným slotom v bloku, alebo presmerovaním z obrazovky editácie bloku na obrazovku pridania slotu s možnosťou zadania konkrétneho požadovaného času.

4.2.4 Navigácia a nastavenia

Hamburger menu a možnosť návratu

Bočné vysúvacie menu je prístupné výlučne z hlavnej obrazovky (*home screen*) prostredníctvom ikony v ľavom hornom rohu. Na všetkých ostatných obrazovkách je namiesto neho zobrazená šípka späť, ktorá používateľa vráti o úroveň vyššie. Keďže aplikácia nie je hlboko vnorená, niekoľkými krokmi späť sa používateľ vždy rýchlo dostane na hlavnú obrazovku, odkiaľ má opäť prístup k menu. Toto riešenie vychádza z heuristiky kontroly a slobody používateľa popísanej v sekcii 4.1 – používateľ má vždy k dispozícii jasný a predvídateľný spôsob návratu. Menu obsahuje nasledujúce položky: *Home*, *Join Room*, *Create Room* (len Majiteľ), *Provide Feedback* a *Settings*. Položka *Logout* je zámerne zvýraznená červenou farbou ako deštruktívna akcia, čím sa predchádza náhodnému odhláseniu.

Nastavenia a správa profilu

Obrazovka nastavení (obrázok 4.7) je dostupná obom rolám a obsahuje nasledujúce možnosti:

- **Edit profile:** Úprava mena, priezviska a dôvodu návštevy, ktorý sa Majiteľovi zobrazuje pri rezervovaných slotoch.
- **Visibility:** Možnosť skryť vlastné meno voči ostatným používateľom z dôvodov ochrany osobných údajov (GDPR). Ako prirodzený dôsledok tohto nastavenia používateľ sám neuvidí mená ostatných.
- **Notify me before:** Nastavenie e-mailových notifikácií pred začiatkom konzultácie. Určuje počet hodín pred začiatkom konzultácie, kedy má byť upozornenie odoslané.
- **Theme:** Prepínač vizuálneho motívu aplikácie.
- **Create Teacher:** Funkcia dostupná výlučne pre Majiteľa, ktorá umožňuje udeliť rolu Majiteľa inému používateľovi.

4.3 Návrh softvérovej architektúry

Aby bol zdrojový kód udržateľný, testovateľný a pripravený na budúce rozširovanie, bolo kľúčové zvoliť vhodnú softvérovú architektúru, ktorá striktne oddeľuje používateľské rozhranie od biznis logiky. Výber architektonického vzoru MVVM a jeho teoretické porovnanie s vzormi MVC a MVP je podrobne zdôvodnené v kapitole 3.2.

4.3.1 Adresárová štruktúra a pomenovacie konvencie

Pre zabezpečenie dlhodobej udržateľnosti kódu, vysokej čitateľnosti a jednoduchšej orientácie v projekte pre prípadných ďalších vývojárov, boli už vo fáze návrhu definované striktné pravidlá pre organizáciu súborov a pomenovacie konvencie (Naming Conventions). Projekt Office Hours sa pri ich definovaní riadil oficiálnymi odporúčaniami jazyka Dart známymi ako *Effective Dart Guidelines* [5]:

- **Triedy, modely a enumeračné typy:** Využívajú formát **UpperCamelCase** (tiež známy ako **PascalCase**). Každé slovo v názve vrátane prvého začína veľkým písmenom bez akýchkoľvek oddeľovacích znakov. Príklad: **CreateBlockViewModel** alebo **ApiService**.
- **Premenné, konštanty a inštancie objektov:** Využívajú formát **camelCase**. Prvé slovo začína malým písmenom a každé ďalšie veľkým. Príklad: **selectedRoomId** alebo **isTeacher**.
- **Súbory a adresáre:** Pre názvy samotných súborov v súborovom systéme bol striktné zvolený formát **snake_case**, kde sú všetky písmená malé a slová sú oddelené podčiarkovníkom. Príklad: **create_block_viewmodel.dart**.

Samotná logická adresárová štruktúra projektu (nachádzajúca sa v primárnom koreňovom priečinku **lib/**) bola priamo odvodená od zvolenej architektúry MVVM. Rozdelenie do logických modulov zaručuje, že vývojár vždy presne vie, kam má umiestniť nový kód:

- **models/:** Priečinok obsahujúci všetky dátové triedy slúžiace na mapovanie JSON odpovedí zo servera do vnútorných štruktúr jazyka Dart.
- **views/:** Priečinok obsahujúci výlučne prezentačnú vrstvu aplikácie. Nachádzajú sa tu jednotlivé samostatné obrazovky (Pages) a podpriečinok **custom_widgets/**, v ktorom sú uložené znovupoužiteľné vizuálne komponenty.
- **viewmodels/:** Každá komplexná obrazovka má tu svoj zodpovedajúci **ViewModel**, ktorý pre ňu spravuje stav a komunikuje so sieťovou vrstvou.
- **services/:** Priečinok združujúci rozličné služby. Obsahuje **api_service.dart** pre sieťovú komunikáciu, **user_preferences.dart** pre prístup k lokálnemu úložisku a tak tiež súbory pre centrálnu správu navigácie, akými sú **app_router.dart** a služby navigátora, ktoré riadia smerovanie a plynulé prechody medzi jednotlivými obrazovkami aplikácie.
- **utils/:** Zložka vyhradená pre statické pomocné triedy, definície globálnych konštánt (**constants.dart**) a validátory používateľských vstupov.

4.4 Možnosti rozšírenia

Architektúra aplikácie Office Hours bola od počiatku navrhovaná s ohľadom na dlhodobú udržateľnosť a prípadné predanie kódu ďalej budúcim vývojárom.

4.4.1 Pripravenosť na platformu iOS

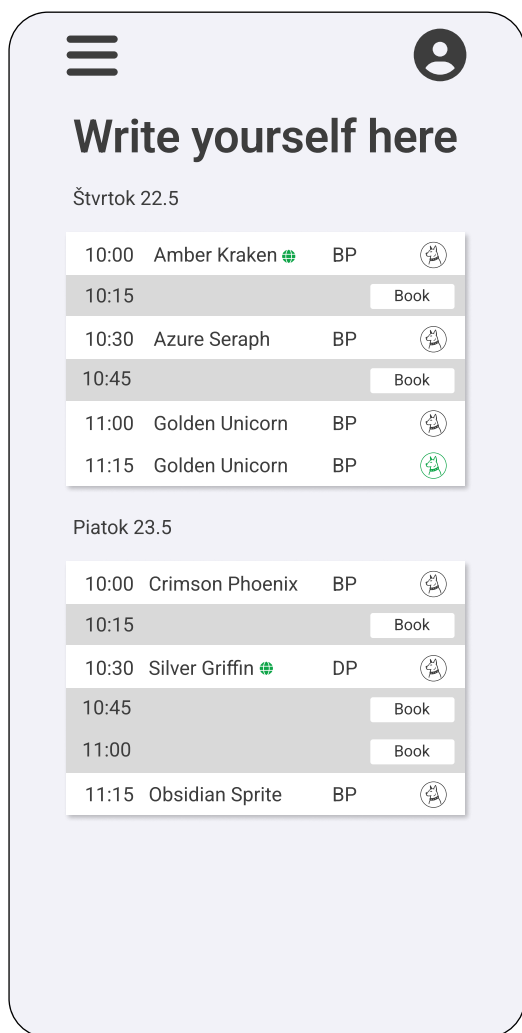
Jedným z hlavných dôvodov pre výber frameworku Flutter bola jeho schopnosť kompilovať jedinou spoločnú bázu kódu do natívnych aplikácií pre viaceré operačné systémy. Vzhľadom na hardvérové obmedzenia počas tvorby tejto práce (absencia platformy macOS potrebnej pre kompiláciu cez vývojové prostredie Xcode) bola aplikácia nasadená, testovaná a publikovaná primárne pre platformu Android.

Zdrojový kód je pripravený aj na bežanie v operačnom systéme iOS a budúci vývojár môže tento existujúci kód s minimálnymi úpravami publikovať priamo do obchodu App Store. Tento prístup šetrí potenciálnym budúcim vývojárom hodiny času, ktoré by inak boli potrebné na vývoj samostatnej aplikácie napríklad v jazyku Swift.

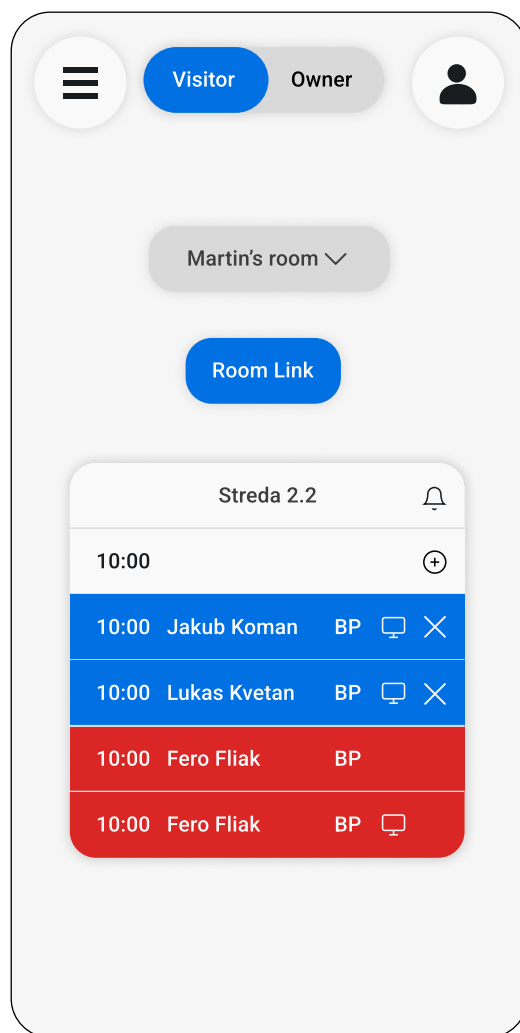
4.4.2 Ďalšie rozšírenia

Možným rozšírením by bola integrácia **push notifikácií**, ktorá by aktívne upozornila študenta na blížiaci sa rezervovaný termín alebo na jeho zrušenie učiteľom. Prínosná by bola aj **integrácia s kalendárom** zariadenia, kde by aplikácia po potvrdení rezervácie automaticky vytvorila udalosť vrátane polohy miestnosti, čím by sa eliminovalo riziko zabudnutia termínu. **Offline režim** s lokálnou vyrovnávacou pamäťou by zase umožnil zobrazenie naposledy načítaných dát aj bez aktívneho internetového pripojenia.

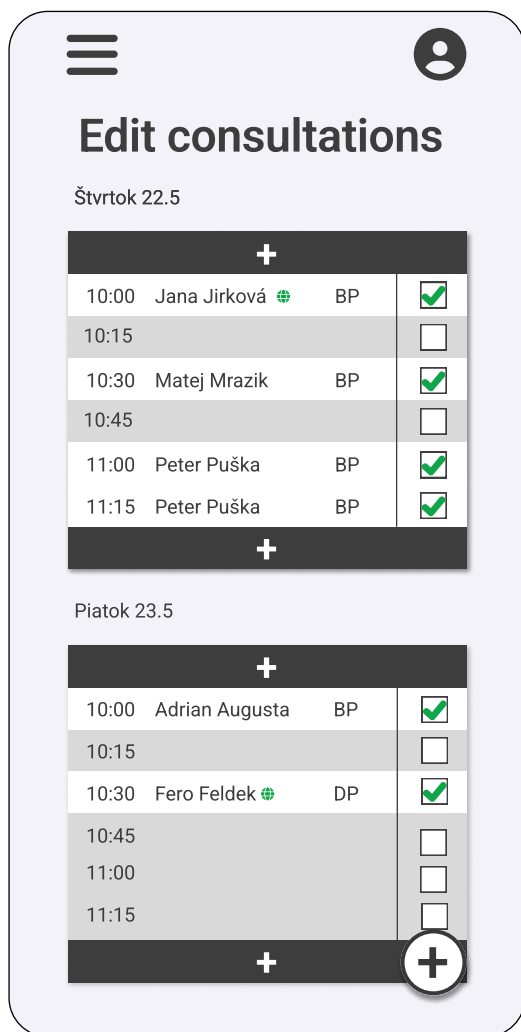
Ďalším prínosom by bola **integrácia s aplikáciou VUT**, ktorá by umožnila zobrazovať potvrdené rezervácie priamo v rozvrhu študenta vedľa bežných predmetov, čím by vznikol jednotný prehľad akademických povinností na jednom mieste. Užitočným doplnkom by bola aj podpora **profilovej fotografie** používateľa, ktorá by učiteľovi uľahčila identifikáciu študentov v zozname rezervácií. V neposlednom rade by **mesačný prehľad rezervácií v kalendári** vizuálne zvýraznil dni s aktívnou rezerváciou, čo by používateľovi umožnilo okamžitú orientáciu v jeho rezerváciách.



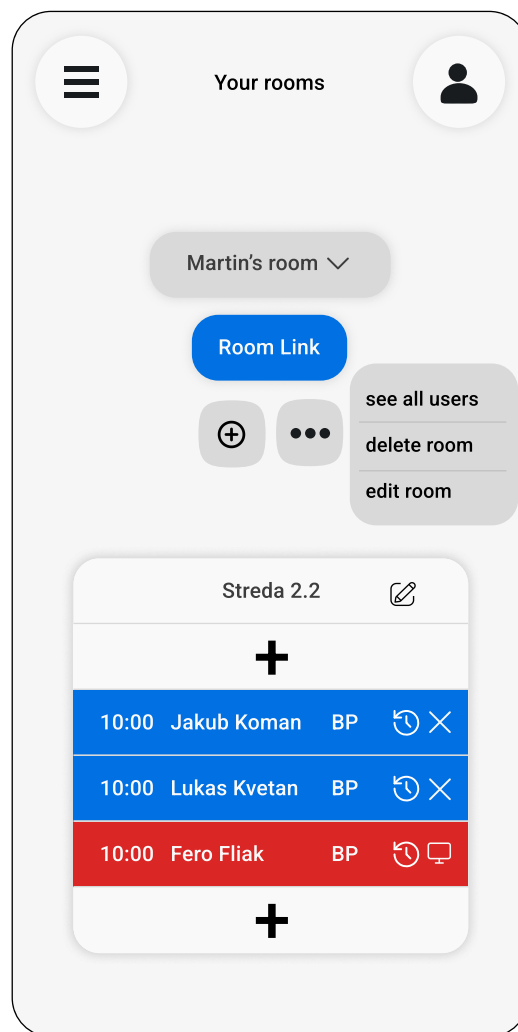
Obr. 4.2: Prvý prototyp domovskej obrazovky Návštevníka s tlačidlami Book a ikonkami strážneho psa.



Obr. 4.3: Druhý prototyp domovskej obrazovky Návštevníka po iteratívnych úpravách: odstránené tlačidlá Book, zjednodušené ikonky a pridaný prepínač medzi rolami.



Obr. 4.4: Prvý prototyp domovskej obrazovky Majiteľa s checkboxmi a bez tlačidiel na správu.



Obr. 4.5: Druhý prototyp domovskej obrazovky Majiteľa s farebnými slotmi, presunutým tlačidlom plus a pridanými nástrojmi na správu bloku a miestnosti.

The image displays two mobile application prototypes side-by-side. The left prototype, titled 'Create block', features a header with a back arrow and a user icon. Below the title, there is a 'Select date' field with a calendar icon, followed by 'Start time' and 'Duration' fields with up/down arrows. Below these are 'Slots' and 'End time' fields, a 'Note' text area, and a checked 'Online' checkbox. A large blue 'Create' button is at the bottom. The right prototype, titled 'Create Room', has a similar header. It includes 'Title', 'Link', and 'Description' text fields. Below these is an 'Allowed emails' section with a text input containing 'user@email.com...' and a plus icon. A link 'See added emails' is positioned above a large blue 'Create' button at the bottom.

<

👤

Create block

Select date

📅

Start time

⬆️⬆️

Duration

⬆️⬆️

Slots

End time

Note

☒ Online

Create

<

👤

Create Room

Title

Link

Description

Allowed emails

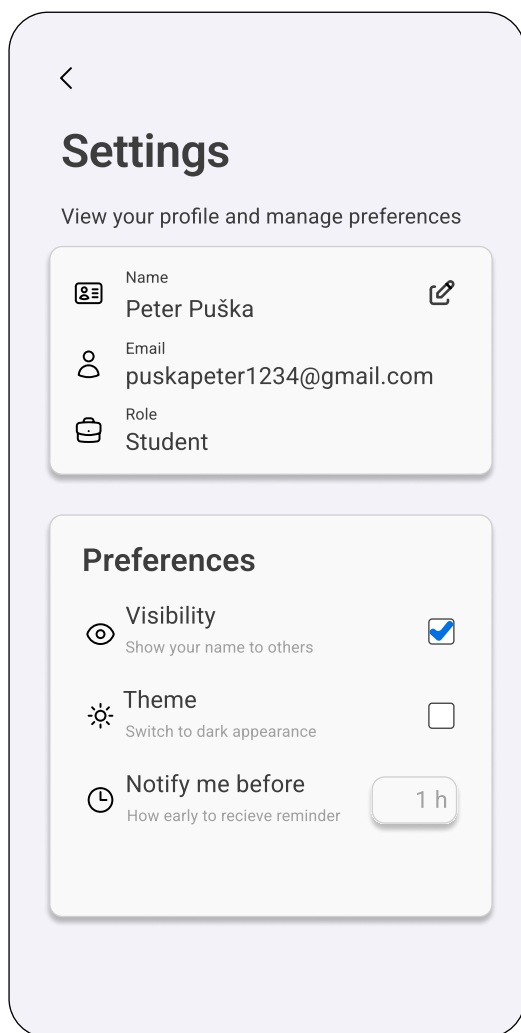
user@email.com...

+

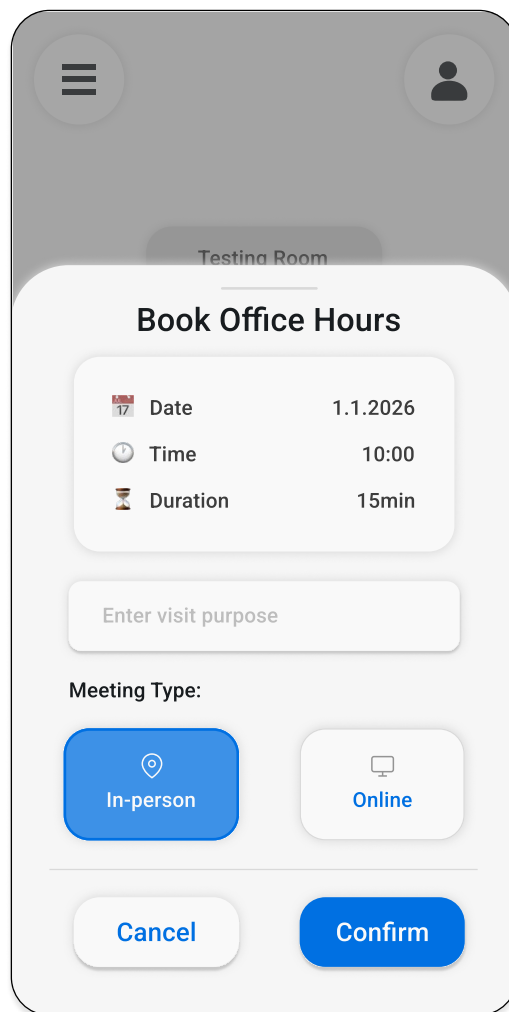
See added emails

Create

Obr. 4.6: Prototypy obrazoviek na vytvorenie bloku (naľavo) a miestnosti s možnosťou dynamického pridávania e-mailov (napravo).



Obr. 4.7: Prototyp znázorňujúci obrazovku nastavení Návštevníka. Majiteľ by mal možnosť navyše vytvoriť nového učiteľa.



Obr. 4.8: Prototyp znázorňujúci dialóg na rezervovanie slotu s dôvodom návštevy a prepínačom online/offline.

Kapitola 5

Implementácia mobilnej aplikácie vo frameworku Flutter

Táto kapitola opisuje konkrétnu realizáciu aplikácie Office Hours vo frameworku Flutter. Nadväzuje na kapitoly 2 a 4 a ukazuje, ako boli požiadavky a návrh používateľského rozhrania a architektúry implementované.

5.1 Prehľad projektu a adresárová štruktúra

Celý zdrojový kód klientskej aplikácie sa nachádza v priečinku **lib/**. Vstupným bodom aplikácie je súbor **main.dart**. Súbor **setup.dart** centralizuje globálne inšancie služieb (navigátor, správca relácie, notifikácie a pod.) dostupné v celej aplikácii. Adresárová štruktúra priečinka **lib/** priamo odráža architektonický vzor MVVM popísaný v kapitole 3. Celková štruktúra projektu je uvedená v prílohe A. Konvencie pomenovania súborov sú popísané v sekcii 4.3.1.

5.2 Architektúra MVVM v praxi

V tejto sekcii je popísané, ako je architektonický vzor MVVM z kapitoly 3 implementovaný v konkrétnych triedach aplikácie.

5.2.1 Dátové modely a JSON konverzia

Na to, aby aplikácia dokázala efektívne pracovať s dátami prijímanými z REST API vo formáte JSON, bolo potrebné navrhnuť silne typované klientske dátové modely. Každý model implementuje metódu `fromJson` na konverziu JSON odpovede na typovanú inštanciu triedy.

Aplikácia pracuje so štyrmi modelmi, ktoré odrážajú štruktúru dát na serveri:

- **UserModel**: Reprezentuje informácie o prihlásenom používateľovi, vrátane jeho mena, priezviska, roly (učiteľ/študent) a e-mailovej adresy.
- **RoomModel**: Reprezentuje konzultačnú miestnosť. Okrem iného obsahuje názov, voliteľný popis – typicky fyzická poloha miestnosti na fakulte, napríklad „L224“ – a pole **link**, ktoré uchováva externý odkaz priradený k miestnosti, napríklad link na

online konzultácie alebo osobnú stránku vyučujúceho. Model ďalej obsahuje zoznam e-mailových adries a domén, ktorým Majiteľ povolil prístup do miestnosti.

- **BlockModel**: Definuje časový blok viazaný na konkrétnu miestnosť. Obsahuje napríklad dátum konania, voliteľnú poznámku, príznak online konania a zoznam inštancií **SlotModel**.
- **SlotModel**: Predstavuje konkrétny rezervovateľný termín. Obsahuje čas začiatku, trvanie v minútach, príznak online konania a pole **takenBy** (ak je slot voľný, hodnota je **null**, ak je obsadený, obsahuje identifikátor používateľa, ktorý si ho rezervoval) a rôzne ďalšie.

5.2.2 Obrazovky a komponenty

Každá obrazovka aplikácie je implementovaná ako samostatný **StatefulWidget** alebo **StatelessWidget**, ktorý zodpovedá jednému súboru `*__page.dart`. Prezentačná vrstva je od logiky oddelená prostredníctvom balíčka **provider** v súlade s architektonickým vzorom MVVM popísaným v kapitole 3: každá obrazovka obaľuje telo widgetom **ChangeNotifierProvider** a reaktívne vykresľovanie zabezpečuje **Consumer<T>**.

Rozlišujú sa dva spôsoby poskytnutia ViewModelu. Ak obrazovka ViewModel vytvára a vlastní, použije sa **ChangeNotifierProvider(create: __)** a životný cyklus inštancie je viazaný na životný cyklus daného **State**. Príkladom je **RegistrationPage** alebo **EmailInputPage**. Ak naopak ViewModel vznikol mimo stromu widgetov – napríklad v metóde **initState()**, použije sa variant **ChangeNotifierProvider.value**, ktorý inštanciu iba sprístupní bez toho, aby ju po zrušení widgetu zničil. Tento prístup je uplatnený napríklad v **ConsultationsOwnerPage** alebo **DisplaySlotHistoryPage**.

Rozdiel medzi oboma prístupmi ilustruje nasledujúca ukážka. Pri prvom prístupe **ChangeNotifierProvider** obrazovka ViewModel vytvára a vlastní. Jeho životný cyklus je viazaný na životný cyklus daného **State**:

```
// ViewModel vytvara a vlastni sama obrazovka (napr. RegistrationPage)
ChangeNotifierProvider(
  create: (_) => RegistrationViewModel(),
  child: Consumer<RegistrationViewModel>(
    builder: (context, viewModel, child) {...
  ),
)
```

Pri druhom prístupe **ChangeNotifierProvider.value** ViewModel vznikol mimo stromu widgetov – napríklad v metóde **initState()**. Provider ho iba sprístupní bez toho, aby ho po zrušení widgetu zničil:

```
// initState: pouzije sa existujuci VM alebo sa vytvori novy
_viewModel = widget.viewModel ?? BaseConsultationsViewModel();
if (widget.viewModel == null) _viewModel.init();

// build: VM sa iba poskytne stromu widgetov, neznici sa s widgetom
return ChangeNotifierProvider.value(
  value: _viewModel,
  child: Scaffold(
```

```

        body: Consumer<BaseConsultationsViewModel>(
            builder: (context, viewModel, child) { ... },
        ),
    ),
);

```

Osobitným príkladom znovupoužiteľnosti je dvojica **CreateRoomPage** a **EditRoomPage**. Oba formuláre zdieľajú rovnaký komponent **RoomFormBody**, ktorý prijíma ViewModel typovaný dynamicky, čím umožňuje jeho použitie s dvoma rôznymi ViewModeli (**CreateRoomViewModel** a **EditRoomViewModel**). Obrazovka **CreateRoomPage** je implementovaná ako **StatelessWidget**, pretože pri vytváraní miestnosti nie je potrebné načítavať žiadne existujúce dáta. Naproti tomu **EditRoomPage** je **StatefulWidget**, ktorý v metóde **initState()** okamžite vyvolá načítanie dát zo servera pomocou kaskádového operátora (**EditRoomViewModel()..loadData(widget.roomId)**).

Hlavná obrazovka pre učiteľa, **ConsultationsOwnerPage**, obaľuje zdieľaný komponent **BaseConsultationsPage** a rozhoduje, ktoré ovládacie prvky komponent odovzdá. Učiteľovi sú prostredníctvom parametrov **addButton**, **deleteButton**, **editBlockButton**, **showHistoryOption**, **addSlotBefore** a **addSlotAfter** predané konkrétne widgety a callbacky. V prípade, že učiteľ prepne pohľad na režim návštevníka pomocou **AnimatedToggle**, sú tieto parametre nahradené hodnotou **null** a **BaseConsultationsPage** vykreslí totožné rozhranie bez akýchkoľvek ovládacích prvkov Majiteľa. Toto riešenie eliminuje duplicitu kódu a zaručuje vizuálnu konzistentnosť.

5.2.3 ViewModely a správa stavu

Každý ViewModel v aplikácii rozširuje triedu **ChangeNotifier** a obsahuje tri spoločné skupiny členov: stavové premenné, gettery a metódy. Stavové premenné sú privátne a prístupné výlučne cez gettery, čo zabráňuje priamej mutácii stavu z prezentačnej vrstvy. Typickými stavovými premennými sú **__isLoading**, **__hasError** a špecifické dáta, napríklad zoznam miestností či slotov. Metóda **notifyListeners()** je volaná vždy na konci každej operácie, ktorá mení stav, čím sa zabezpečí prekreslenie príslušnej časti UI.

Keďže niektoré ViewModely zdieľajú spoločnú funkcionality, bola zavedená hierarchia tried. Abstraktná trieda **BaseRoomViewModel** centralizuje správu formulárových kontrolérov a zoznam povolených e-mailových domén spoločný pre **CreateRoomViewModel** aj **EditRoomViewModel**. Konkrétne podtriedy dopĺňajú iba akcie špecifické pre daný prípad: **CreateRoomViewModel** implementuje metódu **createRoomPage()**, zatiaľ čo **EditRoomViewModel** pred zobrazením formulára načíta existujúce dáta metódou **loadData()** a odlišuje stav počiatočného načítania (**isLoading**) od stavu ukladania zmien (**isSaving**).

Podobná hierarchia je uplatnená aj pri hlavných obrazovkách konzultácií. **OwnerConsultationsViewModel** rozširuje **BaseConsultationsViewModel** a prepisuje metódy **init()** a **fetchRooms()**, čím pridáva paralelné načítanie dvoch zoznamov miestností – pre režim Majiteľa a Návštevníka. Stav prepnutého pohľadu je uložený v premennej **ownerView** a metóda **toggleView()** po zmene hodnoty synchronizuje aktívny zoznam miestností a obnoví dáta vybranej miestnosti.

Viacero ViewModelov využíva vzor optimistickej aktualizácie používateľského rozhrania. Pred odoslaním požiadavky na server sa lokálny stav okamžite zmení a zavolá sa **notifyListeners()**, čím UI reaguje bez viditeľného oneskorenia. V prípade zlyhania API volania sa stav obnoví do pôvodnej hodnoty pomocou vopred uloženého snapshotu. Tento prístup

je uplatnený napríklad v **EditBlockViewModel** pri mazaní slotu alebo prepínaní typu konzultácie, ale aj v **ChangeSettingsViewModel** pri aktualizácii mena, viditeľnosti či nastavenia notifikácií.

Konkrétnym príkladom je metóda pre odrezervovanie slotu **releaseSlot()** v triede **BaseConsultationsViewModel**. Metóda pred odoslaním požiadavky uloží aktuálny stav slotu do premennej **oldSlot** a okamžite aktualizuje lokálnu kópiu s hodnotou **takenBy: null**, čím slot vizuálne zobrazí ako voľný. Až následne prebehne skutočné volanie API. V prípade chyby sa pôvodný stav obnoví z uloženého snapshotu.

```
final oldSlot = _findSlot(slotId);           // 1. ulozi snapshot
_updateSlotInCache(slotId, SlotModel(
    ...
    takenBy: null,                           // 2. okamzita zmena UI
));
try {
    await api.releaseSlot(slotId);
} catch (_) {
    _updateSlotInCache(slotId, oldSlot);      // 3. rollback pri chybe
}
```

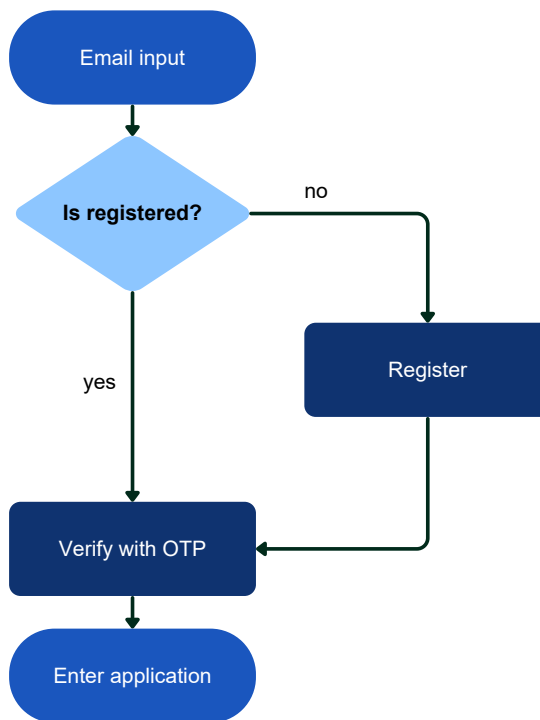
Osobitným prípadom je **EditBlockViewModel**, ktorý prepisuje metódu **notifyListeners()** a podmieňuje jej volanie privátnym príznakom **__disposed**. Toto opatrenie zamedzuje pádu aplikácie v situáciách, keď asynchrónna operácia dobehne po tom, čo bol príslušný widget zrušený.

5.3 Implementácia autentifikácie

Autentifikácia v aplikácii Office Hours je postavená na kombinácii jednorazového hesla (OTP) a JWT tokenu. Celý tok začína na úvodnej obrazovke **EmailInputPage**, kde používateľ zadá svoju e-mailovú adresu. Pred odoslaním požiadavky je adresa overená triedou **Validator** pomocou regulárneho výrazu a príznak **isLoading** vo ViewModeli zabráňuje duplicitnému odoslaniu pri rýchlom opakovanom ťuknutí. Pre prípad, že server neodpovie včas, je spustený päťsekundový časovač, ktorý po uplynutí automaticky odblokuje UI.

Po odoslaní e-mailovej adresy metóda **api.requestLoginOtp()** zavolá server, ktorý vyhodnotí, či používateľ existuje. Odpoveď spracuje metóda **helpers.handleServer()** podľa HTTP stavového kódu: kód 200 znamená, že používateľ existuje a bol mu odoslaný OTP kód – aplikácia ho presmeruje priamo na obrazovku **VerifyOtpPage**. Kód 400 vrátený serverom signalizuje neregistrovanú adresu. Hoci by bol z hľadiska HTTP sémantiky vhodnejší kód 404, toto správanie je dané implementáciou serverovej časti a klient ho spracuje presmerovaním používateľa na **RegistrationPage**, kde vyplní meno, priezvisko a dôvod návštevy, a až po úspešnej registrácii pokračuje na **VerifyOtpPage**. Obe vetvy toku teda končia rovnako – zadaním OTP kódu.

Na obrazovke **VerifyOtpPage** používateľ zadá štvorciferný jednorazový kód doručený na jeho e-mailovú adresu. Prepínač *Remember me* na **EmailInputPage** určuje, či bude token platný 30 minút alebo 90 dní – táto voľba je odoslaná na server spolu s OTP kódom a server podľa nej vydá token s príslušnou dobou expirácie. Pri úspešnom overení je JWT token uložený do šifrovaného úložiska **SecureUserStorage** a necitlivé údaje relácie do **UserPreferences**. Podrobnejší popis oboch úložísk je uvedený v sekcii 5.4.



Obr. 5.1: Vývojový diagram toku autentifikácie v aplikácii Office Hours. Diagram znázorňuje jednotlivé kroky procesu od spustenia aplikácie cez zadanie e-mailovej adresy až po overenie jednorazovým OTP kódom a udelenie prístupu k dátam systému.

Správu relácie počas behu aplikácie zabezpečuje trieda **SessionManager**, ktorá rozširuje **ChangeNotifier** a uchováva stav prihláseného používateľa v pamäti. Pri štarte aplikácie metóda **sm.checkIfInSharedPreferences()** overí, či existuje uložený token, a pomocou balíčka **jwt_decoder** skontroluje jeho expiráciu. Ak je token platný, používateľ je automaticky presmerovaný na hlavnú obrazovku zodpovedajúcu jeho roli. Ak token expiroval, **SessionManager** vymaže reláciu a presmeruje používateľa na prihlasovaciu obrazovku. Rovnaká kontrola prebieha aj počas behu aplikácie prostredníctvom metódy **sm.checkIfValidToken()**, ktorá obsahuje ochranu proti súbežnému spusteniu viacerých odhlasovacích volaní naraz. JWT token je pri každom volaní REST API vkladáný do hlavičky **Authorization: Bearer <token>** – táto mechanika je bližšie opísaná v sekcii 5.4.

5.4 Komunikácia s API a perzistencia dát

5.4.1 Trieda ApiService

Všetka sieťová komunikácia aplikácie je sústredená v triede **ApiService**. Trieda je organizovaná do piatich logických sekcií: autentifikácia, používatelia, miestnosti, bloky a sloty. Každé volanie využíva balíček **http** a prístupuje k rovnakému základnému URL uloženému v triede **Constants**.

Spoločná pomocná metóda **__headers()** zostavuje HTTP hlavičky pre každú požiadavku. Okrem **Content-Type: application/json** vkladá hlavičku **Authorization: Bearer <token>** s aktuálnym JWT tokenom načítaným zo **SessionManager**. Tým je ove-

renie identity zaistené pre každé chránené volanie bez toho, aby ho bolo potrebné riešiť na strane jednotlivých ViewModelov. Spracovanie neautorizovaných odpovedí je centralizované v metóde **__checkUnauthorized()**, ktorá je volaná bezprostredne po každej HTTP odpovedi. Ak server vráti kód 401, metóda automaticky presmeruje používateľa na prihlasovaciu obrazovku. Ostatné chybové kódy sú riešené lokálne: nenaplnené podmienky úspechu vedú k vyhodeniu výnimky **Exception**, ktorú zachytí try-catch blok vo volajúcom ViewModeli. Výsledky úspešných volaní sú deserializované z JSON formátu a namapované na typované modelové triedy prostredníctvom metódy **fromJson**.

5.4.2 Perzistencia dát a ukladanie relácie

Aplikácia rozdeľuje perzistenciu dát do dvoch vrstiev podľa citlivosti informácií. Necitlivé údaje relácie – e-mailová adresa, rola, meno, dôvod návštevy, nastavenie témy a notifikácií – sú ukladané prostredníctvom triedy **UserPreferences**, ktorá obaľuje balíček **shared_preferences**. Metóda **saveItem()** dynamicky zvolí správny setter (**setString**, **setBool** alebo **setInt**) podľa typu hodnoty, čo umožňuje jednotné rozhranie pre ukladanie rôznych dátových typov. JWT token je ukladaný oddelene v triede **SecureUserStorage** využívajúcej balíček **flutter_secure_storage** s Android Keystore, pričom je prístupný výlučne cez metódy **saveToken()**, **getToken()** a **removeToken()**.

Obe úložiská sú zastrešené triedou **SessionManager**, ktorá pri štarte aplikácie načíta celú reláciu do operačnej pamäte metódou **load()** a počas behu ju udržiava synchronizovanú s perzistentným úložiskom prostredníctvom individuálnych aktualizčných metód ako **updateFullName()** či **updateVisibility()**. Vďaka tomu ViewModely prístupujú k údajom relácie priamo z pamäte bez asynchrónnych čítaní z disku.

5.4.3 Spracovanie chýb a validácia vstupov

Validácia formulárových vstupov je sústredená v triede **Validator**. Metóda **validateEmail()** overuje prítomnosť znaku @ a neprázdnych reťazcov na oboch stranách pomocou regulárneho výrazu a pri neúspešnej validácii zobrazí toast so zrozumiteľnou chybovou správou. Metóda **validateNotEmpty()** overuje neprázdnosť hodnoty a o zobrazenie správy sa stará volajúci kód.

Chyby vznikajúce pri sieťovej komunikácii sú zachytávané priamo vo ViewModeloch. Každé volanie **ApiService** je obalené blokom try-catch. Pri zachytení výnimky ViewModel resetuje príznak **isLoading** a prostredníctvom triedy **NotifyUserUtils** zobrazí toast pomocou balíčka **fluttertoast**. Trieda **NotifyUserUtils** poskytuje jednotné rozhranie **showToast()**, ktoré na základe parametra **isError** prepína farebnú schému správy – chybové hlásenia sú zobrazené s červeným pozadím, informatívne s neutrálnym.

5.5 Implementácia konzultácií: miestnosti, bloky a sloty

Táto sekcia opisuje implementáciu konzultačných miestností, blokov a slotov – od zobrazenia a rezervácie termínov až po ich vytváranie a úpravu.

5.5.1 Hlavná obrazovka konzultácií a jej rozšírenie pre Majiteľa

Základom oboch hlavných pohľadov (Majiteľ aj Návštevník) je obrazovka **BaseConsultationsPage**. Ide o **StatefulWidget**, ktorý buď vytvorí vlastný **BaseConsultationsViewModel**, alebo preberie už existujúci ViewModel od volajúceho (typicky z **Consul-**

tationsOwnerPage). Po inicializácii obalí ViewModel pomocou **ChangeNotifierProvider.value** a vykreslí spoločné rozhranie: hornú lištu **AppBarMenu**, bočné menu **SliderMenu** a obsah, ktorý sa nachádza v triede **ConsultationsContent**.

V hornej časti obrazovky je umiestnený komponent **RoomSelectorButton**, ktorý je vložený do widgetu **StickyHeader**. Vďaka tomu zostáva výber miestnosti vizuálne prilepený k vrchu obrazovky pri skrolovaní zoznamu blokov. Pod ním nasleduje obsah konzultácií. Ak nie je vybratá žiadna miestnosť alebo používateľ nemá k dispozícii žiadnu miestnosť, zobrazuje sa komponent **NoRoomsFound** s prispôbenou správou. Študent je nasmerovaný na **JoinRoomPage**, zatiaľ čo Majiteľ je vyzvaný na vytvorenie novej miestnosti cez **CreateRoomPage**.

ConsultationsOwnerPage poskytuje **BaseConsultationsPage** okrem ViewModelu aj množinu voliteľných widgetov a callbackov reprezentujúcich funkcie Majiteľa: tlačidlo **AddBlockButton** na vytváranie blokov, rozbaľovaciu ponuku **SettingsDropDownButton** na správu miestnosti (zobrazenie pripojených používateľov, úprava a zmazanie), tlačidlo **ShowHistoryButton** pre zobrazenie histórie slotov, tlačidlo **EditBlockButton** a dvojicu **AddSlotOnOutskirts** na pridanie slotu na začiatok alebo koniec bloku (obrázok 5.2). Prepínač **AnimatedToggle** umožňuje Majiteľovi prepnúť sa do návštevníckeho pohľadu. V tomto režime odovzdáva **ConsultationsOwnerPage** všetky tieto parametre ako **null**, čím **BaseConsultationsPage** vykreslí rozhranie bez administrátorských akcií.

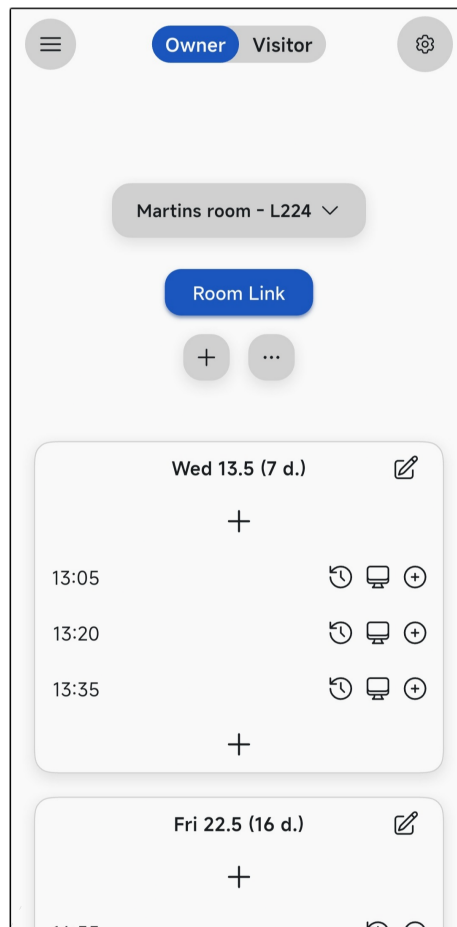
5.5.2 Zobrazenie blokov a slotov, rezervácia a zrušenie slotu

Samotný zoznam blokov v aktuálne vybranej miestnosti je implementovaný v triede **ConsultationsContent**. Táto trieda najprv vykreslí tlačidlo **Room Link**, ktoré otvára externý odkaz priradený k miestnosti a voliteľný riadok s tlačidlami pre Majiteľa (vytvoriť blok, nastavenia miestnosti). Nasleduje buď text „No upcoming consultations found.“, ak miestnosť neobsahuje žiadne bloky, alebo zoznam blokov zoradených chronologicky podľa dátumu. Každý blok je reprezentovaný kartou **ConsultationBlockCard**.

ConsultationBlockCard zobrazuje hlavičku s dátumom bloku a akciou v pravom hornom rohu. V študentskom pohľade je to ikona zvončeka, ktorá umožňuje prihlásenie alebo odhlásenie z e-mailových notifikácií pre daný blok. V pohľade Majiteľa je táto ikona nahradená tlačidlom na úpravu bloku cez **editBlockButton**. Pod hlavičkou sa nachádza voliteľné tlačidlo na vloženie slotu na začiatok zoznamu (iba pre Majiteľa), zoznam slotov a tlačidlo na pridanie slotu na koniec bloku. Tlačidlo na pridanie slotu má ikonku „plus“ a počas odosielania požiadavky na rezerváciu sa mení na animovaný indikátor. Zoznam slotov je vykreslený pomocou **List View.builder**. Každý slot je zabalený v **Selector**, ktorý sleduje dvojicu hodnôt **isTakingSlot** a **isOptimisticallyReleased**. Tým sa zabezpečí, že pri rezervácii alebo uvoľnení slotu sa prekreslí iba daný slot, nie celý zoznam.

O samotné vykreslenie slotu sa stará widget **SlotWidget**. Na základe dát z **SlotModel** a stavu vo ViewModeli najprv určí, či je slot voľný (**takenBy == null** alebo prebieha optimistické uvoľnenie), rezervovaný prihláseným používateľom, alebo rezervovaný iným študentom. Podľa toho zvolí jeden z troch špecializovaných widgetov: **CurrentUserSlot**, **FreeSlot** alebo **AnotherUsersSlot** – ich vizuálne rozlíšenie je popísané v kapitole 4. Widget **CurrentUserSlot** navyše obsahuje tlačidlo na uvoľnenie, ktoré pri zrušení rezervácie v krátkom časovom predstihu zobrazí varovný dialóg.

SlotWidget zároveň vlastní dve spodné karty (bottom sheets). Pri ťuknutí na voľný slot otvorí **TakeSlotBottomSheet** s obsahom navrhnutým v kapitole 4. Po potvrdení zavolá callback **onTakeSlot()** s poznámkou a príznakom online a okamžite kartu zavrie.

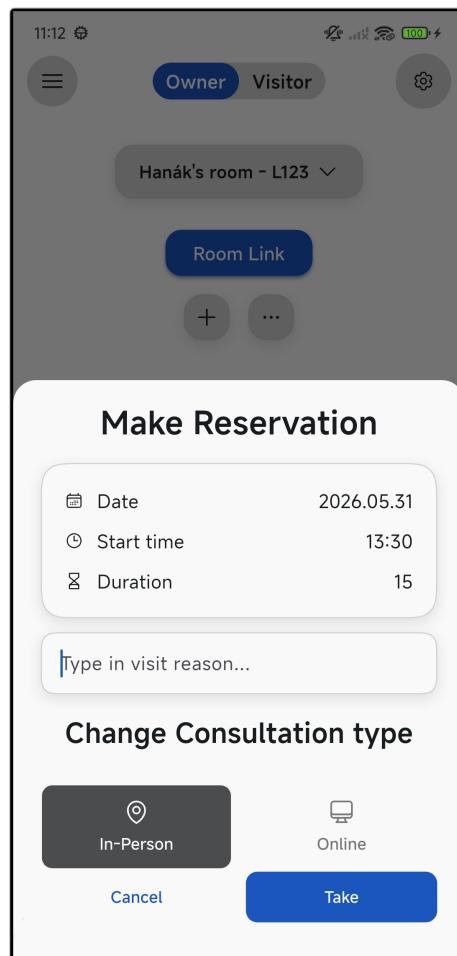


Obr. 5.2: Hlavná obrazovka konzultácií v pohľade Majiteľa. V hornej lište je viditeľný prepínač *Owner/Visitor* implementovaný ako **AnimatedToggle**. Pod ním je prilepený **RoomSelectorButton** s názvom aktuálne vybranej miestnosti. Tlačidlá „+“ a „...“ slúžia na vytvorenie nového bloku a správu miestnosti. Každý blok zobrazuje dátum s relatívnym označením počtu dní a ikonu úpravy vpravo hore. Sloty vo vnútri bloku obsahujú čas začiatku a akčné ikony pre históriu, typ konzultácie a pridanie slotu.

Pri kliknutí na obsadený slot môže byť zobrazená **TypeBottomSheet**, ktorá umožňuje Majiteľovi zmeniť typ konzultácie bez rušenia rezervácie a návštevníkovi zmeniť typ jeho vlastnej konzultácie.

5.5.3 Pripojenie do miestnosti (JoinRoomPage)

Obrazovka **JoinRoomPage** umožňuje študentom vyhľadať a pripojiť sa k ľubovoľnej dostupnej miestnosti. Po inicializácii **ViewModel** načíta zoznam všetkých miestností, ktoré sú študentovi dostupné podľa nastavených e-mailových domén a adries. Horná časť obrazovky obsahuje formulár s komponentom **Autocomplete**, ktorý nad zoznamom miestností ponúka dynamické dopĺňanie podľa zadaného textu. Zoznam možností sa filtruje podľa podreťazca v názve miestnosti a miestnosti, do ktorých je študent už prihlásený, sú z ponuky automaticky skryté.



Obr. 5.3: Spodná karta **TakeSlotBottomSheet** zobrazená po kliknutí na voľný slot. Obsahuje prehľad vybraného termínu (dátum, čas, trvanie), textové pole pre zadanie dôvodu návštevy a prepínač typu konzultácie (*In-Person/Online*). Tlačidlo „Take“ potvrdí rezerváciu a kartu okamžite zavrie.

Okrem výberu zo zoznamu **Autocomplete** priebežne kontroluje aj manuálne zadanie názvu – ak používateľ zadá presný názov existujúcej miestnosti bez toho, aby položku vybral zo zoznamu, ViewModel vyhodnotí zhodu a nastaví vybranú miestnosť. Po stlačení tlačidla „Join room“ sa študent pokúsi do zvolenej miestnosti pripojiť. V spodnej časti stránky je zoznam miestností, do ktorých je študent už pripojený.

5.5.4 Vytvorenie a editácia bloku

Vytváranie nových blokov je realizované na obrazovke **CreateBlock**. Formulár pokrýva parametre bloku definované v návrhu (kapitola 4). Výber dátumov zabezpečuje dialóg **CustomDateRangePickerDialog**, ktorý podporuje výber jedného dňa, rozsahu alebo viacerých nesúvislých dní podľa aktuálne zvoleného režimu. Zvolený rozsah je zobrazený v textovej podobe priamo v hlavnom formulári a pri potvrdení je uložený do ViewModelu.

Časová konfigurácia je riešená pomocou komponentu **CustomTimeDurationPicker**. Po zadaní počtu slotov ViewModel vypočíta koncový čas bloku a zobrazí ho v neklikateľnom poli **End Time** (obrázok 5.5). Tlačidlo „Create“ po stlačení zavolá metódu **createB-**



Obr. 5.4: Tri vizuálne stavy widgetu **SlotWidget** v rámci jedného bloku. Červený slot predstavuje variant **AnotherUsersSlot** – termín rezervovaný iným študentom. Modrý slot zodpovedá variantu **CurrentUserSlot** – termín prihláseného používateľa s tlačidlom na zrušenie rezervácie. Biely slot je voľný termín (**FreeSlot**) s ikonou na pridanie slotu za neho. Skratky pri slotoch označujú dôvod návštevy študenta.

lock(roomId, onSuccess), ktorá po úspešnom vytvorení bloku spustí callback **onSuccess** - typicky **viewModel.loadRoom()** na **ConsultationsOwnerPage** - a tým obnoví zoznam blokov v pôvodnej obrazovke.

Úprava existujúceho bloku prebieha na obrazovke **EditBlockPage**. Po načítaní bloku **ViewModel** zobrazí názov miestnosti, dátum bloku a zoznam slotov v komponentoch **Slot-Row**. Každý riadok obsahuje čas slotu, poznámku o obsadenosti, ikonku online/offline a tlačidlo na mazanie. Okrem toho obrazovka ponúka funkciu **copyBlock**, ktorá umožňuje skopírovať celý blok vrátane rozloženia slotov na iné dátumy opäť pomocou dialógu **CustomDateRangePickerDialog**. Prepínač „Online“ na úrovni bloku umožňuje hromadne prepnúť typ všetkých slotov (napríklad na online z dôvodu choroby), pričom po úspešnom uložení sa opäť volá callback **onSuccess** pre obnovu rodičovskej obrazovky (obrázok 5.6).

5.5.5 Vytvorenie a editácia miestnosti

Vytváranie miestnosti je realizované obrazovkou **CreateRoomPage**, ktorá je implementovaná ako **StatelessWidget**. Využíva spoločný komponent **RoomFormBody** pokrývajúci všetky atribúty miestnosti, definované v návrhu (kapitola 4). Po stlačení tlačidla „Create“ sa zavola metóda **createRoom()**, ktorá vstupy odošle na server (obrázok 5.7).

EditRoomPage zdieľa rovnaký formulár **RoomFormBody**, avšak ako **StatefulWidget** v metóde **initState()** najprv načíta existujúce dáta miestnosti z API a predvyplní nimi príslušné textové polia a zoznam **acceptedEmails**.

5.5.6 Prehľad používateľov a história slotu

Z pohľadu Majiteľa sú k dispozícii dve doplnkové obrazovky poskytujúce detailnejší prehľad o používaní miestnosti. **DisplayUsersInRoomPage** zobrazuje zoznam všetkých používateľov, ktorí sa pripojili do danej miestnosti. Obrazovka po načítaní dát zobrazí informáciu o prázdnom zozname, alebo zoznam e-mailových adries v tvare kariet. V hornej časti sa zároveň zobrazuje počet členov miestnosti.

História konkrétneho slotu je prístupná z pohľadu Majiteľa cez tlačidlo **ShowHistory-Button** na každom slote. Po stlačení sa otvorí **DisplaySlotHistoryPage**, ktorá vizuálne reprezentuje chronologický zoznam všetkých rezervácií daného slotu, vrátane informácie, kto a kedy slot obsadil alebo uvoľnil. Týmto spôsobom má Majiteľ k dispozícii možnosť spätne overiť históriu rezervácií daného slotu.

5.5.7 Prehľad vlastných rezervácií

Obrazovka **MyReservationsPage** bola pridaná na základe spätnej väzby testerov, ktorí postrádali centrálny prehľad vlastných rezervácií naprieč všetkými miestnosťami. Obrazovka je prístupná z hamburger menu a zobrazuje všetky aktívne rezervácie prihláseného používateľa bez ohľadu na to, do ktorej miestnosti patria.

Obrazovka je implementovaná ako **StatelessWidget**, ktorý pri zostrojení vytvorí inštanciu **MyReservationsViewModel** a okamžite spustí inicializáciu kaskádovým volaním **MyReservationsViewModel()..initialize()**. Metóda **initialize()** vykoná dve sieťové volania: **api.getMyReservations()** na získanie rezervovaných slotov a **api.getAllRooms()** na získanie zoznamu miestností, ktoré sú uložené v premennej **__cachedRooms**. Pomocná metóda **slotEssentials(index)** pre každý slot vráti mapu zobraziteľných hodnôt: názov miestnosti, dátum, čas začiatku, trvanie, poznámku a príznaky prebiehajúcich operácií. Kým prebieha načítavanie, zobrazuje sa indikátor. Po jeho dokončení sa vykreslí zoznam kariet, alebo prázdny stav **__EmptyState** s výzvou na rezervovanie prvého termínu.

Každá karta umožňuje dve akcie. Klepnutím na ikonu krížika používateľ zruší rezerváciu – metóda **releaseSlot()** slot okamžite odstráni z lokálneho zoznamu a skutočné volanie API prebehne až následne. Klepnutím na ikonu obrazovky alebo polohy sa otvorí **ConsultationTypeBottomSheet**, kde používateľ potvrdí zmenu typu konzultácie – metóda **changeType()** analogicky prehodí príznak **is_online** v lokálnej kópii ešte pred odpoveďou servera. V prípade zlyhania ktorejkoľvek operácie sa stav obnoví z vopred uloženého snapshotu a používateľovi sa zobrazí chybový toast. Keďže obe akcie môžu prebiehať súčasne na rôznych kartách, ViewModel sleduje ich stav v mapách **__optimisticallyReleased** a **__changingType** s kľúčom podľa identifikátora slotu.

5.5.8 Obrazovka nastavení (ChangeSettings)

Obrazovka nastavení je implementovaná ako **StatelessWidget**, pričom **ChangeSettingsViewModel** sa vytvára kaskádovým volaním **ChangeNotifierProvider(create: (__) => ChangeSettingsViewModel()..initialize())**. Metóda **initialize()** sa teda spustí ihneď po konštrukcii ViewModelu a asynchrónne stiahne aktuálne nastavenia používateľa zo servera. Kým prebieha načítavanie, obrazovka zobrazuje indikátor načítavania. Po jeho dokončení sa vykreslí pomocný widget **__SettingsBody**, do ktorého sa deleguje celé rozloženie. Obrazovka je rozčlenená do troch kariet.

Prvá karta zobrazuje **profil používateľa**: e-mailovú adresu a rolu, ktoré sú iba na čítanie a zobrazujú sa pomocou **InfoTile** a editovateľné položky mena, priezviska a dôvodu návštevy (Visit Reason), ku ktorým sa pristupuje cez **EditableTile**. Po klepnutí na ikonu ceruzky sa otvorí príslušný dialóg, ktorý po potvrdení zavolá metódu ViewModelu a zmenu synchronizuje so serverom.

Druhá karta obsahuje **predvoľby**. Prvou z nich je prepínač **Visibility**, ktorý riadi GDPR flag viditeľnosti mena voči ostatným používateľom. Tento flag sa prenáša do API pri každej jeho zmene cez metódu **viewModel.setVisibility()** a jeho aktuálna hodnota sa zároveň uloží do **UserPreferences** pre okamžité lokálne prekreslenie. Vypnutie viditeľnosti má obojstranný dôsledok – používateľ skryje svoje meno ostatným, no ako protiľahu zároveň prestane vidieť mená iných používateľov v slotoch. Druhá predvoľba, **Notify me before**, umožňuje nastaviť, koľko hodín vopred má systém odoslať e-mailové upozornenie pred začiatkom rezervovanej konzultácie. Pôvodná implementácia pracovala s jednou hodnotou, avšak na základe spätnej väzby od testerov, ktorí požadovali možnosť byť upozornení viackrát pred tým istým termínom, bola nahradená zoznamom hodnôt.

Predvoľba je implementovaná ako **NotifyHoursTile**. Používateľ zadá počet hodín do číselného vstupného poľa a hodnotu potvrdí kliknutím na ikonu plus vpravo od poľa, čím sa zavolá **viewModel.addNotifyHour(parsed)**. Takto možno postupne zostaviť zoznam viacerých časových odstupov, napríklad 24 hodín a 2 hodiny pred konzultáciou. Aktuálny počet pridaných hodnôt je zobrazený ako odkaz *View added hours (n)*, ktorý presmeruje na samostatnú obrazovku so zoznamom, kde je možné jednotlivé záznamy odstrániť volaním **viewModel.removeNotifyHour()**. Prázdny zoznam notifikácie deaktivuje. Každá zmena zoznamu je odoslaná na server volaním **api.updateNotificationTimes()** a pri zlyhaní sa zoznam obnoví zo snapshotu pomocou rovnakého vzoru optimistickej aktualizácie, aký je použitý inde v aplikácii. Poslednou predvoľbou je **Theme**: prepínač medzi svetlou a tmavou tematikou. Na rozdiel od ostatných nastavení sa táto zmena neprenáša na server, ale ukladá výlučne lokálne do **UserPreferences**, pretože ide o čisto vizuálnu preferenciu zariadenia.

Tretia karta, **Administration**, sa zobrazí podmienene len vtedy, keď má ViewModelom načítaná rola hodnotu **'teacher'**. Obsahuje jedinú akciu **Create Teacher**, ktorá otvorí dialóg pre zadanie e-mailu, mena a priezviska nového pedagóga.

5.5.9 Znovupoužiteľné komponenty nastavení

Nastavenia sú oddelené do dvoch samostatných súborov: **settings_tiles.dart** a **settings_dialogs.dart**.

Súbor **settings_tiles.dart** definuje päť typov dlaždíc.

- **InfoTile**: Riadok iba na čítanie. Zobrazuje popisok a hodnotu.
- **EditableTile**: Rozširuje **InfoTile** o ikonu ceruzky, ktorá cez callback **onEdit** otvorí príslušný dialóg.
- **CheckboxTile**: Riadok s vlastným prepínačom **CustomCheckbox** vpravo. Ikona sa dynamicky mení podľa aktuálneho stavu (napríklad **eye_open/eye_closed** pre Visibility, **moon/sun** pre Theme).
- **ActionTile**: Kliknuteľný riadok so šípkou vpravo, určený pre akciu Create Teacher.
- **NotifyHoursTile**: Jediný **StatefulWidget**, pretože spravuje vlastný **TextEditingController** pre číselné vstupné pole. Metóda **didUpdateWidget** zabezpečuje synchronizáciu poľa s hodnotou ViewModelu v prípade, že sa zmení externe (napríklad po potvrdení zo servera). Ikonka sa mení na preškrtnutú pri hodnote 0.

Súbor **settings_dialogs.dart** obsahuje tri samostatné asynchrónne funkcie – **showNameDialog**, **showVisitReasonDialog** a **showCreateTeacherDialog**. Dialógy pre editáciu mena a dôvodu návštevy sú implementované ako jednoduché **AlertDialog** widgety so zdieľanými pomocnými funkciami **__cancelButton** a **__saveLabel**, čím je zabezpečená vizuálna konzistencia akčných riadkov naprieč všetkými dialógmi. Dialóg Create Teacher navyše využíva **StatefulBuilder** na správu lokálneho stavu načítavania a chybovej správy priamo v dialógu bez nutnosti povýšenia volajúceho widgetu na **StatefulWidget**.

5.6 Navigácia a prepojenie obrazoviek

Navigácia v aplikácii je centralizovaná v triede **NavigationService**, ktorá je registrovaná ako singleton prostredníctvom balíčka **get_it** [3] a sprístupňuje sa globálne cez inštan-

ciu **nav**. Táto trieda obaľuje štandardné metódy Flutteru (**Navigator.push**, **Navigator.pop**), pričom interne udržiava **routeHistory** (zásobník názvov navštívených obrazoviek). Ten umožňuje **NavigationService** rozhodnúť, ktorú obrazovku zobraziť pri stlačení tlačidla späť. Pre každú obrazovku vystavuje pomenovanú navigačnú metódu, napríklad **nav.toCreateBlock(...)**. Vďaka tomu nie je v žiadnom ViewModeli ani widgete priamo referencovaná žiadna konkrétna trieda obrazovky, čo znižuje závislosť medzi vrstvami a uľahčuje prípadné budúce zmeny v navigačnom grafe a zároveň robí kód prehľadnejším.

5.6.1 AppBar a hamburger menu

Obrazovky aplikácie využívajú zdieľaný widget **AppBarMenu** ako hodnotu parametra **appBar** v **Scaffold**. Jeho obsah sa mení podľa kontextu: na hlavných obrazovkách sa vpravo zobrazuje ikona s tromi čiarami, otvárajúca bočné menu **SliderMenu** s navigačnými položkami, zatiaľ čo na vnorených obrazovkách je nahradená šípkou späť volajúcou **nav.pop()**. V strede AppBaru sa na hlavnej obrazovke Majiteľa nachádza prepínač **AnimatedToggle** pre prepínanie medzi rolami a odkaz na nastavenia je zobrazený len na obrazovkách, kde to je relevantné. Na samotnej stránke nastavení sa vďaka parametru **onSettingsPage: true** skryje.

5.6.2 Navigačné metódy a spätné volanie **onSuccess**

Na niektorých obrazovkách sa používa vzor odovzdania callbacku **onSuccess**. Každá takáto obrazovka deklaruje parameter **final VoidCallback? onSuccess** a po úspešnom dokončení operácie ho zavolá ešte pred **nav.pop()**. Napríklad **AddSlotPage** (obrázok 5.8) má signatúru:

```
class AddSlotPage extends StatefulWidget {  
  final int blockId;  
  final VoidCallback? onSuccess;  
  ...  
}
```

a pri navigácii rodič odovzdá vlastnú metódu ViewModelu:

```
nav.toAddSlot(  
  blockId: widget.blockId,  
  onSuccess: () => viewModel.refetchData(widget.blockId),  
);
```

Keď potomok zavolá **widget.onSuccess?.call()** ešte pred **nav.pop()**, rodič si znovunačíta dáta zo servera skôr, ako sa obrazovka zobrazí. Návrat je teda vždy do aktuálneho stavu, napríklad **viewModel.loadRoom()** pri prechode na **CreateBlockPage** alebo **EditBlockPage**, alebo **viewModel.refetchData(blockId)** pri prechode na **AddSlotPage**.

5.7 Znovupoužiteľné widgety a vizuálne prvky

Táto sekcia opisuje widgety zdieľané naprieč viacerými obrazovkami aplikácie, rozdelené podľa adresárovej štruktúry.

Priečinkok custom_widgets/

- **AnimatedToggle:** Animovaný prepínač s *pill* animáciou. Používa sa na prepínanie medzi rolami Majiteľa a Návštevníka.
- **AppBarMenu:** Zdieľaný panel implementovaný ako **PreferredSizeWidget**. Podľa kontextu zobrazuje hamburger alebo šípku späť, ikonu nastavení a voliteľný **AnimatedToggle**.
- **SliderMenu:** Bočné menu odovzdávané do parametra **drawer**. Dáta načítava cez vlastný **SliderMenuViewModel**. Položka Create Room je viditeľná len pre Majiteľa.
- **CustomInputTextField:** Konfigurovateľné textové pole so vstavanou správou životného cyklu kontroléra a podporou typov **text**, **email**, **url** a **number**.
- **CustomDateRangePickerDialog:** Dialóg pre výber dátumov s režimami *multiple* a *range*. Používa sa pri vytváraní a kopírovaní blokov.
- **CustomTimeDurationPicker:** Dvojica **CupertinoPicker** komponentov pre výber trvania slotu a času začiatku. Používa sa na obrazovke **CreateBlockPage** a **AddSlotPage**.
- **RoomFormBody:** Zdieľaný formulár pre **CreateRoomPage** a **EditRoomPage** prijímajúci ViewModel typovaný ako **BaseRoomViewModel**.
- **CustomCheckbox:** Obal nad štandardným **Checkbox** widgetom so zväčšením dotykovej plochy o 20 %.

Priečinkok base_consultations/

RoomSelectorButton zabezpečuje výber aktívnej miestnosti a zostáva prilepený k vrchu obrazovky pri skrolovaní pomocou **StickyHeader**. **NoRoomsFound** zobrazuje prispôbenú správu a akciu podľa roly. Widgety **ConsultationBlockCard** a **ConsultationContent** sú opísané v sekcii 5.5.2.

Priečinkok slots/

Hlavné widgety slotov (**SlotWidget**, **FreeSlot**, **CurrentUserSlot**, **AnotherUsersSlot**, **TakeSlotBottomSheet**, **TypeBottomSheet**) sú opísané v sekcii 5.5.2. **TypeBottomSheet** umožňuje zmenu typu konzultácie (online / osobne) bez potreby zrušenia rezervácie. Pomocné widgety zahŕňajú **SlotTime** pre zobrazenie času a trvania, prepínač stavu **TypeToggle** a **KeyboardPadding** pre priestor pri zobrazenej klávesnici.

Priečinkok settings/

Widgety v tomto priečinku sú opísané v sekcii 5.5.8.

6:12

Selected: None

Minutes per Slot

5 min

10 min

15 min

20 min

25 min

Block Start Time

16 00

17 05

18 10

19 15

20 20

Slots

End Time

Note

☐ Online

Create

Obr. 5.5: Obrazovka **CreateBlock** s formulárom pre výber dátumu, počtu slotov a trvania pomocou **CustomTimeDuration-Picker**.

6:11

Edit block

in

Hanák's room

Sun 31.5 + Add Slot

14:30

14:45 Taken

15:00 Taken

15:15

15:30

☐ Online

Copy Block

Delete Block

Obr. 5.6: Obrazovka **EditBlockPage** so zoznamom slotov, prepínačom hromadnej zmeny typu konzultácie na úrovni bloku a možnosťami kopírovať a vymazať blok.

6:12

6:12

< ⚙

Create Room

Title (e.g. Martin's room)

https://example.com

Description (e.g. D105)

Cancel deadline in hours

Allowed email domains

@domain.com, user@domain.com.. +

[View added domains \(0\)](#)

Create

Obr. 5.7: Obrazovka **CreateRoomPage** s formulárom **RoomFormBody** pre zadanie názvu, popisu, externého odkazu a zoznamu povolených e-mailových domén.

6:11

6:11

< ⚙

Create Slot

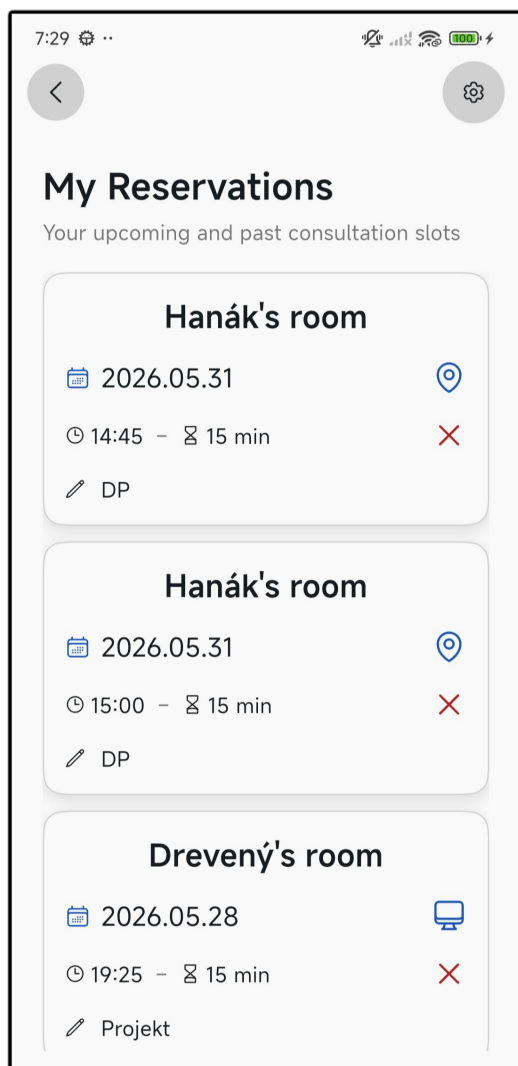
Minutes per Slot	Slot Start Time
5 min	16 00
10 min	17 05
15 min	18 10
20 min	19 15
25 min	20 20

Note

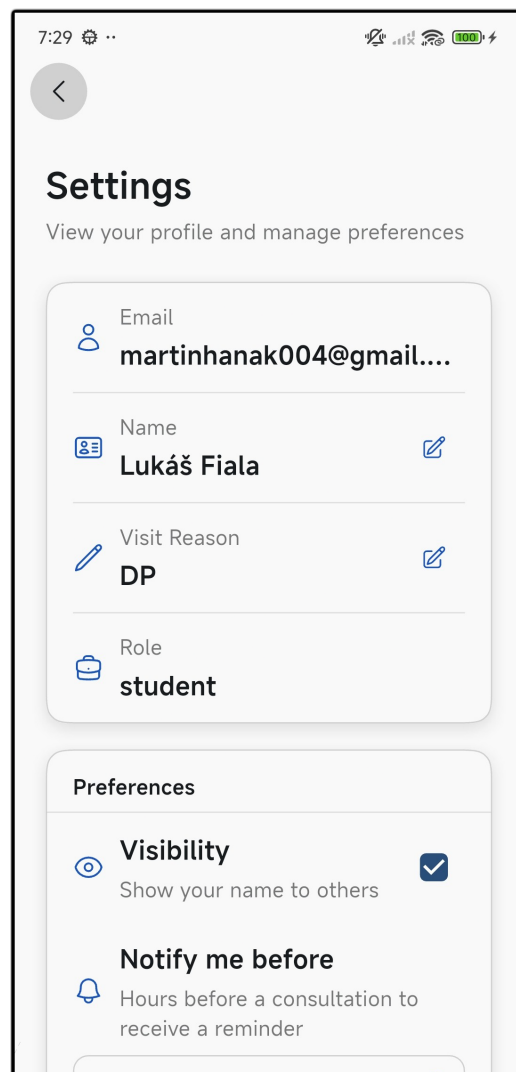
☐ Online

Create

Obr. 5.8: Obrazovka **AddSlotPage** s komponentom **CustomTimeDurationPicker** pre výber času začiatku a trvania nového slotu.



Obr. 5.9: Obrázok **MyReservations-Page** so zoznamom aktívnych rezervácií naprieč všetkými miestnosťami. Každá karta zobrazuje názov miestnosti, dátum, čas začiatku, trvanie a dôvod návštevy. Ikona polohy alebo obrazovky vpravo hore indikuje typ konzultácie a slúži na jeho zmenu. Ikona krížika umožňuje okamžité zrušenie rezervácie s optimistickou aktualizáciou UI.



Obr. 5.10: Obrázok **ChangeSettings** zobrazujúca dve z troch kariet. Prvá karta obsahuje profilové údaje používateľa: e-mailovú adresu (len na čítanie), editovateľné meno cez **EditableTile** a dôvod návštevy. Druhá karta *Preferences* obsahuje prepínač **Visibility** pre GDPR viditeľnosť mena a predvoľbu **Notify me before** pre nastavenie e-mailových upozornení.

Kapitola 6

Testovanie a nasadenie aplikácie

6.1 Manuálne testovanie

Manuálne testovanie prebiehalo na fyzickom zariadení aj na emulátore s cieľom overiť správanie aplikácie v rôznych hardvérových a softvérových konfiguráciách. Konkrétne boli použité nasledujúce konfigurácie:

- Xiaomi Redmi Note 11 Pro 5G (fyzické zariadenie, Android 13),
- Google Pixel 5 (emulátor, Android 11).

Osobitná pozornosť sa venovala hraničným situáciám, ktoré môžu v reálnom použití nastať. Testoval sa výpadok siete počas aktívnej relácie, vstupné polia s neplatnými hodnotami (napríklad neprimerane dlhé reťazce alebo znaky), ktoré nie sú povolené (napr. emoji). Tieto testy odhalili niekoľko okrajových prípadov, ktoré bolo potrebné ošetriť na úrovni validácie formulárov.

6.2 Používateľské testovanie

Používateľské testovanie sa uskutočnilo s cieľom získať spätnú väzbu od reálnych používateľov. Ešte pred formálnym testovaním hotovej aplikácie prebehla neformálna konzultácia prototypov so spolužiakmi, ktorá pomohla odhaliť skoré nejasnosti v návrhu rozhrania a overiť základné koncepty ovládania. Následné testovanie funkčnej aplikácie prebehlo opäť so spolužiakmi a vedúcim záverečnej práce, teda s používateľmi, ktorí by aplikáciu mohli v budúcnosti reálne využívať. Takéto zloženie skupiny testerov umožnilo zachytiť problémy na viacerých úrovniach – od technických nedostatkov až po nejasnosti v používateľskom rozhraní.

6.2.1 Metodológia testovania

Pri osobnom testovaní bola využitá metóda testovania pozorovaním (angl. *observation-based testing*) [14]. Pri tomto type testovania vývojár sleduje vybraných používateľov priamo počas používania aplikácie a zaznamenáva ich správanie bez aktívneho zasahovania. Sleduje sa, či sa používateľ vie plynulo pohybovať po aplikácii, či nachádza požadované funkcie bez váhania a či mu spôsobuje neistotu niektorý prvok rozhrania. Výhodou tejto metódy je, že odhaľuje skutočné problémy s použiteľnosťou, ktoré si vývojár po dlhodobej práci na projekte sám nevšimne. Testovanie pozorovaním bolo doplnené krátkym rozhovorom po skončení každého sedenia, počas ktorého tester vyjadril celkový dojem a návrhy na zlepšenie.

6.2.2 Priebeh testovania

Testovanie prebiehalo v dvoch paralelných formách. Osobné testovanie s vybranými testermi zahŕňalo samostatnú registráciu každého testera a splnenie konkrétnych scenárov. Pre potreby testovania funkcionality učiteľa bol v API k dispozícii endpoint **GET /users/test-switch-role**, určený výhradne na testovacie účely, ktorý umožnil prepnúť rolu používateľského konta z roly Návštevníka na rolu Majiteľa. Vďaka tomu testerí po štandardnej registrácii získali učiteľské oprávnenia. Testerí riešili nasledujúce scenáre:

- **Rezervovanie termínu:** Tester sa prihlásil do existujúcej miestnosti a zarezervoval si dostupný slot. Scenár overoval správnosť zobrazovania dostupných termínov, funkčnosť výberu typu konzultácie a správne uloženie rezervácie.
- **Vytvorenie miestnosti a bloku termínov:** Pomocou učiteľského konta tester vytvoril novú miestnosť, nakonfiguroval jej nastavenia a vygeneroval blok na niekoľko dní. Scenár overoval validáciu vstupných polí a správne vytvorenie blokov.
- **Zrušenie existujúcej rezervácie:** Tester zrušil predtým vytvorenú rezerváciu. Scenár overoval, či sa slot správne uvoľní a či je zmena viditeľná v reálnom čase.
- **Zmena nastavení profilu a miestnosti:** Tester upravil nastavenia svojho profilu a následne aj nastavenia miestnosti. Scenár overoval správnosť ukladania zmien a ich okamžité premietnutie.

Osobné testovanie prebiehalo na internátnej izbe, kde bol každý tester posadený za počítač so spusteným emulátorom. Počas plnenia scenárov autor sledoval vývojársku konzolu a zaznamenával chyby v reálnom čase. Po skončení scenárov prebehol krátky rozhovor zameraný na celkový dojem a návrhy na zlepšenie. Súbežne s osobným testovaním prebiehalo aj vzdialené testovanie v rámci schvaľovacieho procesu obchodu **Google Play**. Vzdialení testerí mali k dispozícii možnosť *Provide Feedback* v menu aplikácie, ktorá otvorila Google formulár umožňujúci zaslanie spätnej väzby priamo z ich zariadenia.

6.3 Výsledky testovania a implementované zmeny

6.3.1 Získaná spätná väzba

Testerí poskytli pestrú spätnú väzbu. Viacerí sa zhodli na tom, že aplikácia je funkčná, no v niektorých častiach rozhrania pôsobí neintuitívne. Konkrétne postrehy možno rozdeliť do nasledujúcich skupín:

- **Navigácia:** Testerí si všimli, že po návrate do zoznamu miestností sa vždy zobrazí prvá miestnosť, nie tá naposledy navštívená. Navrhli zapamätanie poslednej aktívnej miestnosti.
- **Informovanosť používateľa:** Pri nepripojení do miestnosti neboli zobrazené povolené domény, čo bolo máťúce. Tester nevedel, prečo sa pripojiť nedá.
- **Pomenovanie tlačidiel:** Generické názvy ako „Submit“ či „Confirm“ nepôsobili prirodzene. Testerí navrhli použitie popisnejších, kontextovo vhodnejších názvov. Napríklad „Take“, „Change“ a „Create“.

Tabuľka 6.1: Prehľad zistených problémov a implementovaných zmien

Problém	Kto nahlásil	Implementovaná zmena
Po návrate do zoznamu sa vždy otvorí prvá miestnosť	Vzdialené testovanie	Aplikácia si pamätá naposledy otvorenú miestnosť
Nepripojenie bez zobrazenia povolených domén	Vzdialené testovanie	Pri chybe pripojenia sa zobrazia povolené domény
Page Not Found pri navigácii späť z úvodnej obrazovky	Vzdialené testovanie	Opravené smerovanie navigácie
Chýbal prehľad vlastných rezervácií naprieč miestnosťami	Osobné testovanie	Pridaná obrazovka My Reservations s možnosťou zrušenia a zmeny typu konzultácie
Nevhodné pomenovanie tlačidiel	Osobné testovanie	Tlačidlá premenované na kontextovo výstižnejšie názvy
Predvyplnenie dôvodu návštevy pri zmene slotu	Osobné testovanie	Dôvod návštevy sa predvyplňa z pôvodnej rezervácie
Editovateľný prepínač napriek obmedzeniu učiteľom	Osobné testovanie	Pole je zobrazené ako neaktívne (disabled)
Možnosť nastaviť iba jedno upozornenie pred konzultáciou	Osobné testovanie	Predvoľba <i>Notify me before</i> rozšírená na zoznam hodnôt umožňujúci viacnásobné upozornenia

- **Zmena dôvodu návštevy:** Pri zmene slotu nebol predvyplnený dôvod návštevy, čo testerí považovali za nepohodlné a zbytočne zdĺhavé.
- **Neaktívne prvky:** Keď je spôsob stretnutia implicitne nastavený učiteľom (napr. online), pole sa napriek tomu javilo ako editovateľné. Testerí navrhli, aby bolo viditeľne neaktívne („unclickable“).
- **Chyba pri navigácii:** Pri použití tlačidla späť na úvodnej obrazovke („landing screen“) sa zobrazila stránka „Page Not Found“.
- **Prehľad vlastných rezervácií:** Aplikácia neposkytovala celkový prehľad rezervácií naprieč všetkými miestnosťami na jednom mieste, čo testerí považovali za chýbajúcu funkciu.
- **Upozornenia pred konzultáciou:** Bolo možné nastaviť iba jedno upozornenie pred konzultáciou, čo testerí považovali za obmedzujúce. Navrhli možnosť nastavenia viacerých upozornení.

Na základe spätnej väzby boli problémy identifikované a opravené. Podrobnejší popis je zaznamenaný v tabuľke 6.1.

6.4 Nasadenie aplikácie

6.4.1 Publikácia v obchode Google Play

Na publikáciu aplikácie bolo potrebné vytvoriť vývojársky účet v službe **Google Play Console**, čo zahŕňa jednorazový registračný poplatok (v čase písania 25 USD) [12]. Pred odoslaním do obchodu bol zostavený **release build** pomocou príkazu **flutter build app-bundle --release** a výsledný balíček vo formáte **AAB** (Android App Bundle) bol nahratý do konzoly.

Proces schvaľovania nebol bezproblémový. Keďže vývojársky účet bol vytvorený po 13. novembri 2023, Google vyžaduje pre nové osobné účty uzavreté testovanie s minimálne 12 testerami po dobu 14 dní pred získaním prístupu do produkcie [9]. Prvá žiadosť o postup do produkčného vydania bola preto zamietnutá z dôvodu nedostatočného otestovania aplikácie v uzavretej testovacej fáze. Po predĺžení testovacej periódy a aktívnejšom zapojení testerov bola žiadosť odoslaná druhýkrát a následne úspešne schválená. Aplikácia je sprístupnená v obchode **Google Play**.¹

¹https://play.google.com/store/apps/details?id=com.mhanak.consultation_app.

Kapitola 7

Záver

Táto bakalárska práca si kládla za cieľ navrhnuť a implementovať modernú, bezpečnú a multiplatformovú mobilnú aplikáciu „Office Hours“ pre správu a rezerváciu konzultačných hodín na Fakulte informačných technológií VUT v Brne.

V rámci analýzy boli preskúmané existujúce riešenia – systém Wiki@Merlin, Diplomky FIT, Google Kalendár a Microsoft Excel. Každé z nich pokrýva iba časť požadovanej funkcionality, pričom žiadne neponúka súčasne ochranu rezervácií, mobilnú aplikáciu, kontrolu prístupu, viditeľnosť obsadených slotov, e-mailové notifikácie a možnosť online stretnutia (Tabuľka 2.1). Táto medzera sa stala motiváciou pre vývoj vlastného riešenia.

Na základe identifikovaných nedostatkov boli špecifikované funkčné a nefunkčné požiadavky a navrhnutý diagram prípadov použitia. Používateľské rozhranie vznikalo iteratívne – od ručných náčrtov cez prototyp vo Figma až po finálnu implementáciu, pričom celý proces bol riadený spätnou väzbou. Aplikácia bola implementovaná vo frameworku Flutter s využitím jazyka Dart a architektonického vzoru MVVM, ktorý zaručuje čisté oddelenie prezentačnej vrstvy od biznis logiky. Kľúčové funkcie zahŕňajú autentifikáciu pomocou jednorazového OTP kódu, správu miestností, blokov a slotov, optimistickú aktualizáciu rozhrania a nastavenia súkromia v súlade s GDPR. Aplikácia bola otestovaná manuálne aj používateľsky na emulátore a fyzických zariadeniach.

Stanovené ciele práce boli splnené. Výsledná aplikácia priamo odstraňuje nedostatky analyzovaných systémov: rezervácie sú chránené pred neoprávnenou manipuláciou, prístup k miestnostiam je riadený na úrovni e-mailových adries a domén, obsadené sloty sú pre Návštevníka viditeľné a systém môže aktívne informovať používateľov o zmenách, pokiaľ majú notifikácie povolené. Aplikácia bola navrhnutá a implementovaná ako multiplatformová. Zdrojový kód je plne pripravený na kompiláciu pre iOS, avšak publikácia pre túto platformu zostáva bezprostredným ďalším krokom, keďže počas tvorby práce nebol dostupný počítač s operačným systémom macOS nevyhnutným pre Xcode. Aplikácia bola úspešne nasadená pre platformu Android a publikovaná v obchode Google Play. Zdrojový kód je štruktúrovaný a zdokumentovaný tak, aby umožňoval ľahkú orientáciu a ďalší rozvoj budúci vývojármi. Možné smery ďalšieho rozvoja aplikácie sú podrobnejšie rozpracované v kapitole 4, v sekcii venovanej možnostiam rozšírenia. Súčasťou odovzdaných výstupov je taktiež plagát prezentujúci aplikáciu a krátke video demonštrujúce jej praktické použitie, ktoré boli vytvorené v súlade s požiadavkami zadania.

Literatúra

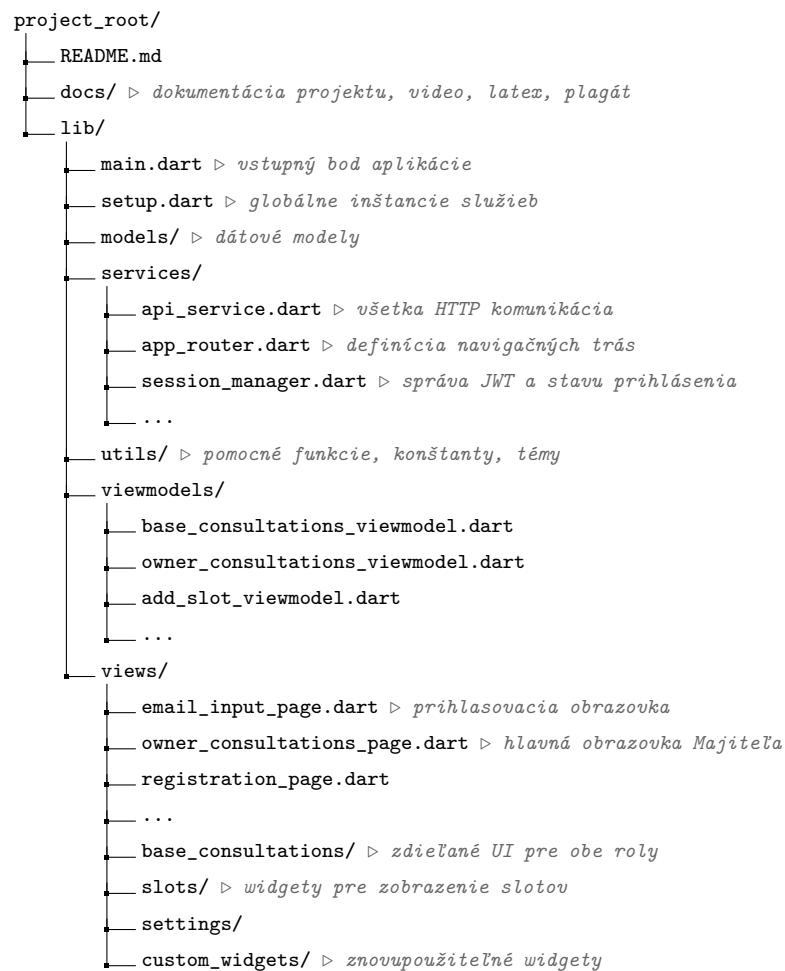
- [1] AKHTAR, N. a GHAFOOR, S. Analysis of Architectural Patterns for Android Development. In: *Proceedings of the International Conference on Computing and Information Technology*. June 2021.
- [2] BHANARKAR, S. *Why the 4-Point Grid System Dominates UI Design*. 2024. Dostupné z: <https://uxplanet.org/why-the-4-point-grid-system-dominates-ui-design-88c2400e775a>.
- [3] BURKHART, T. *Get_it – Flutter package*. 2024. Dostupné z: https://pub.dev/packages/get_it.
- [4] DART TEAM. *Dart Overview*. 2024. Dostupné z: <https://dart.dev/overview>.
- [5] DART TEAM. *Effective Dart: Style*. 2024. Dostupné z: <https://dart.dev/effective-dart/style>.
- [6] DART TEAM. *Sound null safety*. 2024. Dostupné z: <https://dart.dev/null-safety>.
- [7] FLUTTER TEAM. *Building user interfaces with Flutter*. 2024. Dostupné z: <https://docs.flutter.dev/ui>.
- [8] FLUTTER TEAM. *Flutter architectural overview*. 2024. Dostupné z: <https://docs.flutter.dev/resources/architectural-overview>. Oficiálna dokumentácia frameworku Flutter.
- [9] GOOGLE. *App testing requirements for new personal developer accounts*. 2023. Dostupné z: <https://support.google.com/googleplay/android-developer/answer/14151465>.
- [10] GOOGLE LLC. *Flutter – Build apps for any screen*. 2024. Dostupné z: <https://flutter.dev>.
- [11] GOOGLE LLC. *Appointment booking in Google Calendar*. 2025. Dostupné z: <https://workspace.google.com/resources/appointment-scheduling/>.
- [12] GOOGLE LLC. *Get started with Play Console*. 2026. Dostupné z: <https://support.google.com/googleplay/android-developer/answer/6112435>.
- [13] JETBRAINS. *What is cross-platform mobile development?* Kotlin Multiplatform Documentation. 2025. Dostupné z: <https://kotlinlang.org/docs/multiplatform/cross-platform-mobile-development.html>.

- [14] KRUG, S. *Rocket Surgery Made Easy: The Do-It-Yourself Guide to Finding and Fixing Usability Problems*. New Riders, 2010. ISBN 978-0321657299.
- [15] META PLATFORMS, INC.. *About the New Architecture*. 2024. Dostupné z: <https://reactnative.dev/docs/the-new-architecture/landing-page>.
- [16] MICROSOFT CORPORATION. *Create an alert to get notified when a file or folder changes in SharePoint* Microsoft Support. 2025. Dostupné z: <https://support.microsoft.com/en-us/office/create-an-alert-to-get-notified-when-a-file-or-folder-changes-in-sharepoint-e5a79e7b-a146-46da-a9ef-d65409ba8918>.
- [17] NIELSEN, J. Enhancing the explanatory power of usability heuristics. In: ACM. *Proceedings of the ACM CHI'94 Conference on Human Factors in Computing Systems*. Boston, MA: [b.n.], April 1994, s. 152–158.
- [18] PARRISH, M. *Flutter_secure_storage – Flutter package*. 2024. Dostupné z: https://pub.dev/packages/flutter_secure_storage.
- [19] ROUSSELET, R. *Provider – Flutter package*. 2024. Dostupné z: <https://pub.dev/packages/provider>.
- [20] SINHAL, A. *MVC, MVP and MVVM Design Pattern*. 2016. Dostupné z: <https://medium.com/@ankit.sinhal/mvc-mvp-and-mvvm-design-pattern-6e169567bbad>.
- [21] WROBLEWSKI, L. *Mobile First*. 2009. Dostupné z: <https://www.lukew.com/ff/entry.asp?933>.

Príloha A

Adresárová štruktúra projektu

Nasledujúci obrázok zobrazuje adresárovú štruktúru implementovanej aplikácie (`lib/`).



Obr. A.1: Adresárová štruktúra projektu.

Príloha B

Prezentačný plagát



VYSOKÉ UČENÍ
TECHNICKÉ
V BRATISLAVE

FAKULTA
TECHNICKÝCH
INFORMAČNÝCH
TECHNOLÓGIÍ

Mobile Application for Scheduling Consultations

Author: Martin Hanák
Supervisor: prof. Ing. Adam Herout, Ph. D.

Abstract

A mobile app for scheduling academic consultations, built with **Flutter** (MVVM) and secured via **OTP** email verification. It replaces outdated systems by giving students easy **booking and cancellation**, while providing instructors with full control over slots, blocks, and rooms. The app communicates with a **dedicated API**.

Technology Stack

For development, I used **Dart, Flutter, Postman and Figma**



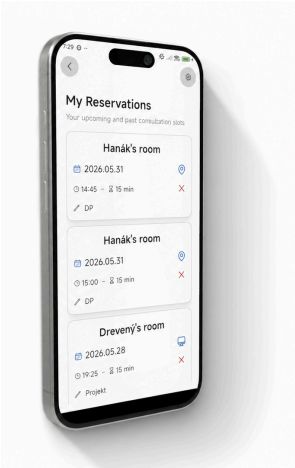
Key Features

For Students

- Browse and join consultation rooms by name
- Book or cancel appointment slots with a note
- Choose online or in-person consultation per slot
- Set notifications for slot availability
- Secure login via OTP email verification

For Teacher

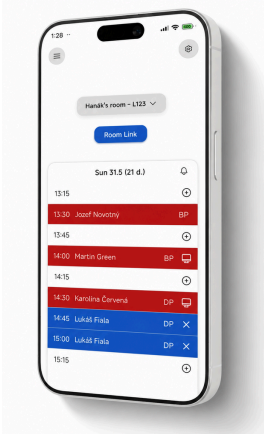
- Create consultation rooms with custom access rules
- Create time blocks for multiple dates at once
- Manage individual slots
- Switch online/offline mode per block or slot
- View slot history and list of joined students
- Automatic email notifications sent to students



Download

From Google Play Store





Obr. B.1: Plagát prezentujúci mobilnú aplikáciu pre správu konzultácií, vytvorený ako súčasť obhajoby bakalárskej práce.